Operating Systems from the Bare Metal Up

Building logOS on a simulated RISC-V computer

Pavel Gladyshev

Sample chapters

Chapter 1: What is an operating system?
Chapter 11: Cooperative multitasking

Draft of November 12, 2026

Copyright © 2026 Pavel Gladyshev. All rights reserved.

This sample contains two chapters of a book in preparation: Chapter 1, the introduction, and Chapter 11, the first chapter of Part III. The table of contents is that of the whole book, and references to other chapters point to chapters that are not in this sample. The text is a draft and may change before publication.

The example programs and circuits are at `github.com/pavelgladyshev/bare-metal-os-supplement`, and logOS itself at `github.com/pavelgladyshev/logos`.

Contents

1 What is an operating system? 1

I The machine 19

2 Programming the RISC-V computer in assembly language 20

3 From source code to a running program 52

4 C on bare metal 82

II The operating system from the outside: logOS Belfield 1.0 110

5 General-purpose operating systems and booting 111

6 File systems 132

7 System calls and the Unix programming interface 164

8 The Unix shell 197

9 Shell scripting 230

10 logOS Belfield 1.0: a minimal operating system with a shell 251

III Doing many things at once: logOS Blackrock 2.0 278

11 Cooperative multitasking 279

12 Computer architecture and interrupts 304

13 Preemptive multitasking 331

14 logOS Blackrock 2.0: processes and multitasking 351

15 Pipes and interprocess communication 385

16 Process scheduling 411

IV Sharing the machine safely: logOS Glendalough 3.0 **430**

17 Synchronisation **431**

18 Semaphores, deadlock and monitors **450**

19 Virtual memory **472**

20 logOS Glendalough 3.0: virtual memory in logOS **494**

21 Multiprocessors **523**

22 Protection and access control **548**

A RISC-V quick reference **574**

B The simulated computer **587**

C Setting up **596**

D Testing logOS **607**

Chapter 1

What is an operating system?

Switch on a laptop and within a minute you can type a letter, play music, browse the web and compile a program, all at once, in windows you drag about with the mouse. Underneath, the machine is the one we can build from logic gates: a processor that fetches an instruction from memory, executes it and fetches the next; some RAM; some ROM; and a handful of devices that appear to the processor as locations in memory. Without a program it is, as I like to put it, a pile of metal. Something has to stand between that pile of metal and the word processor, and that something is the *operating system*. Everyone has used one, most people can name three, and yet the standard textbooks find it surprisingly hard to say exactly what an operating system *is*. Andrew Tanenbaum's *Modern Operating Systems* opens its definition by admitting that it is hard to pin down. So what is it, and why does it deserve a book?

This chapter gives a first answer. It explains why operating systems are worth studying, builds a working definition, separates the two great families of computer systems and of the operating systems that run on them, and follows the history of the ideas from programs written as lists of numbers in the early 1960s to the graphical microcomputers of the 1980s, with the help of four films in which the inventors explain their ideas themselves. It then introduces the *kernel*, the part of the operating system that everything else rests on, and the simulated RISC-V computer and the small operating system, logOS, that we will build and study in the rest of the book. It ends with a guide to the book and its tools.

1.1 Why study operating systems?

When I was a student, about thirty years ago, I sat through lectures on computer architecture, operating systems and assembly language and wondered, as you may be wondering now, whether I would ever need any of it. My own field turned out to be digital forensics, finding evidence in computers for the courts, and it has made me look deeper into computer systems, and at a lower level, than I ever expected. But the real reason is simpler. Operating systems and the machine underneath them are the foundation of our discipline. A doctor who specialises in dermatology still had to learn anatomy and physiology; a software developer who will spend a career writing web applications still needs to know what actually happens when the program opens a file, starts a thread or waits for the network.

Operating systems is a vast subject. In the 1980s it could fill two full semester courses of a computing degree; since then, networking and later artificial intelligence have claimed much of that time, and a modern introduction has to choose. This book chooses three themes.

1. **Hardware and software co-evolved.** Many features of a modern processor exist only because operating systems needed them. People had an idea about how computers should be used; the idea required a change in the hardware; the new hardware in turn made new kinds of software possible. Interrupts, timers, privileged modes of execution, memory protection and virtual memory, atomic instructions — each has a story of this kind, and we will tell it as we meet each feature. Seeing the relationship is, to me, the most interesting part of the subject.

2. **The operating system as the programmer sees it.** As a practitioner you need to know how to use the services a modern operating system provides: processes, pipes, the shell, the file system and the system calls that give access to all of them. We study them mostly in the form Unix gave them, because that form became the common language of operating systems, but the aim is the concepts, not the details of one system.

3. **The operating system from the inside.** Finally, we look under the bonnet to see how those services are realised in software and hardware: device drivers, system-call processing, process scheduling, virtual memory, synchronisation.

The third theme explains the shape of the book. An operating system manages hardware, so it is itself a low-level program running directly on the machine — on the "bare metal". To understand how it works we have to start at the same level: machine code, memory-mapped devices, the stack, the linker. I could talk about processes, priority queues and context switching from the first page, but unless you have programmed a context switch driven by timer interrupts yourself, and watched it run, the rest of the subject risks seeming like religion: a set of things you are asked to believe. So we start with the bare machine and build up, and every mechanism in the book is one you can run.

There is one more reason, which I will come back to when we reach Unix. We are craftspeople: we make software. A painter learns by studying masterpieces, and so should we. Some of the most beautiful pieces of software ever written are operating systems, or parts of them, and reading them develops a sense of what good design looks like that no amount of advice can give.

1.2 A first definition

Ask anyone to name an operating system and the answers come at once: Windows, Linux, macOS. A few more thoughtful ones follow: Android and iOS on phones; the many specialised *real-time operating systems* that run inside cars, aircraft and factory machines; and historic systems that are no longer used, such as IBM's OS/360 for its System/360 mainframes or Microsoft's MS-DOS for the first IBM PCs. Many more will be written in the future. What do they have in common?

1.2.1 An interface

The first thing they share is their purpose. An operating system is an *interface* between the software we want to write and the hardware it runs on, and we want that interface to be the *same* on as many different computers as possible. We write an application for a particular operating system and expect it to run, without modification or almost without, on any computer that runs that operating system: the program that runs on your laptop runs on mine, although the two machines have different

processors, different amounts of memory, different screens and disks from different manufacturers. The operating system gives software a uniform view of hardware that is in reality very varied.

When we write a game, we do not want to care whether the graphics card is made by one company or another. We want a robust, reliable *application programming interface* that we can build the game on, and the operating system provides it. One of the earliest such interfaces, still with us today, is the set of system calls that Unix gave its programs; we will spend a good part of the book on it.

1.2.2 Why "operating", and why "system"?

The name itself says a good deal. *Operating*: the operating system helps the user to operate the computer, and the programmer to program it. *System*: it is not one program but a system of many, which work together. What kinds of programs?

The most important are the *device drivers*. A driver gives software a standard interface to one particular type of hardware — a display, a keyboard, a graphics card, a disk. Each manufacturer builds its device in its own proprietary way, and supplies a piece of software that presents that device through the standard interface. An application then never needs to know whose device it is talking to: it expects an implementation of the standard interface, and uses it to deliver whatever the application is for. Every new device needs a new driver, but none of the applications has to change.

The operating system also comes with *utilities*: the system programs you get on any computer when you install it, such as Notepad and File Explorer on Windows, or `ls`, `cp` and the text editors on Linux. And it comes with a *shell*, the program through which the user starts other programs, whether it is a command line or a desktop full of icons. Underneath all of these is the kernel, which we will meet in Section 1.5. Figure 1.1 shows the arrangement.

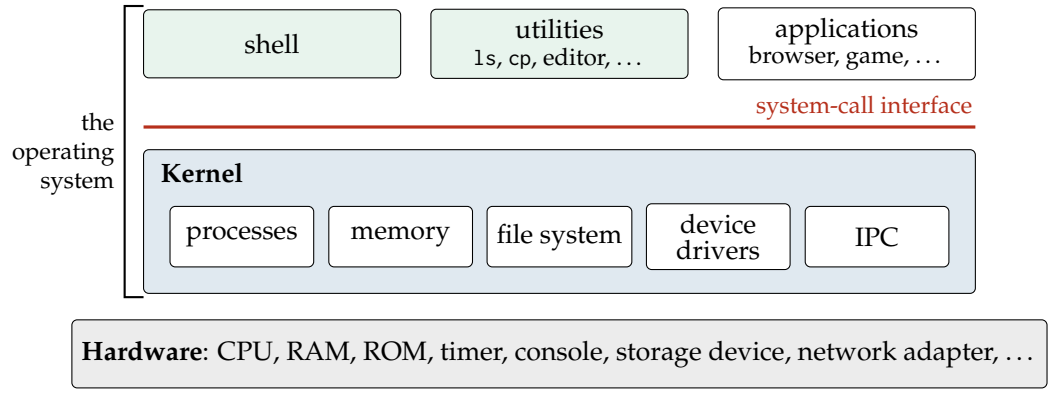


Figure 1.1: An operating system is a system of programs. The kernel, with its drivers, runs directly on the hardware; the shell and the utilities are ordinary programs that come with it. Applications and utilities alike reach the hardware only through the system-call interface.

Where exactly the operating system ends is a matter of taste and, occasionally, of law: in the late 1990s the question whether a web browser was part of Windows ended up in an American court. For us the useful boundary is the system-call interface in Figure 1.1. Everything below it is the kernel; everything above it is an ordinary program, even if it ships with the operating system.

1.2.3 Making computers easier to use

The history of the computer industry, once we knew how to make a computer that works, has been one long effort to make computers easier to use, and operating systems are where most of that effort went. For some thirty years, from the 1950s to the end of the 1980s, a large part of the research and development in computing was about exactly that. It did not happen overnight. There was no single moment when someone walked on stage with a finished personal computer; our understanding of what the technology is, and how it can be used, grew gradually over decades.

What made this possible is, I think, a very human ability: the ability to pretend. We can pretend that the letter x stands for a number and the letter y for another, add them, and get a third letter that stands for their sum. Numbers are themselves a kind of pretence, standing for real objects in the world. Operating systems are built on pretences of this kind, which we call *abstractions*. A disk is a long row of numbered blocks; we pretend it is a cabinet of named folders holding named files. A processor runs one instruction stream; we pretend it runs dozens of programs at once, and call each one a *process*, an instance of a running program. A device is a set of registers at odd addresses; we pretend it is a file we can read and write. Many operating system ideas began as almost philosophical concepts of this kind, and the rest of the work was to make the hardware and software that let us keep up the pretence efficiently and safely.

1.2.4 What the operating system is for

We can now say what an operating system is for. Its purpose is to manage the resources of the computer efficiently and to provide a standard interface that application programs can use. Spelled out a little more, a general-purpose operating system has three main goals.

1. **Reliable and efficient concurrent execution of many processes.** Several programs run at the same time, and the operating system must make that both reliable and efficient. Reliable means *protection*: a mistake in one program must not crash the others, nor the operating system itself. Efficient means using the processor, memory and devices cleverly — an interactive program that needs a quick response should come before a backup that only has to finish eventually. And where there are several users with different rights, the system must *control their access* to its resources.

2. **Interprocess communication.** Processes must be able to cooperate on a common task. A Unix pipeline runs several programs at once and feeds the output of each into the input of the next; a web server runs a separate process for each incoming connection. The operating system provides the means for processes to pass data and to synchronise.

3. **Simpler application development.** Write a program once and run it on every computer the operating system runs on, without caring about the details of the hardware.

Key idea

An operating system is a system of programs that manages a computer's resources and gives application programs a uniform, convenient and safe interface to hardware that is in reality varied, awkward and shared.

1.3 Kinds of computer systems and of operating systems

Every computer we will meet has the same underlying structure: a CPU with one or more cores, RAM, ROM, and devices controlled through memory-mapped registers. That is as true of your laptop as of the controller in a washing machine. But in what they are *for*, computer systems fall into two large families, and their operating systems follow.

1.3.1 Embedded and general-purpose computer systems

Embedded systems are built into other things: lift controllers, washing machines, building access-control systems, cable TV boxes, car engines. They are probably the vast majority of all computers in existence, and they have features that suit bare-metal programming:

- They run a single application that does not change. Nobody updates a washing machine so that it treats your clothes more gently. The program can be written in C and assembly, compiled into a ROM image, programmed into the ROM and manufactured in hundreds of thousands of copies.
- User interaction, if there is any, is specialised and scripted: which buttons to press to select a wash programme.
- Quick response is often essential. A microcontroller running a car or aircraft engine must react to events within a guaranteed time; such a system is called *real-time*.
- The code lives in ROM or flash memory and is rarely, if ever, updated, and usually one company makes both the hardware and the software.

General-purpose computer systems are the ones on your desk and in your pocket: laptops, phones, gaming PCs, servers, supercomputers. They run a wide range of applications, they are heavily interactive, and we want to change the applications often — install a new one, upgrade an old one. So the software is organised differently: the operating system, the application code and the data are stored as files in a *file system* on a block storage device, such as a solid-state drive (in the past a hard disk), and are loaded from there when needed.

1.3.2 Embedded and general-purpose operating systems

The same division runs through operating systems. *Embedded operating systems*, often called real-time operating systems (RTOS) because of the property just mentioned, are tailored to run a single application from a single vendor. The application may still have several activities to juggle. A Mars rover has to drive its motors across the Martian surface, talk to Earth, run its scientific instruments and cameras, all at once; the software is all NASA's, but it consists of several threads of activity that must make progress together. An RTOS is the kind of software that makes this easy. Typically it arrives as a library that we link with our program, providing functions we can call from C to create and destroy processes and to let them synchronise and communicate. There is usually no interactive shell, and the system guarantees a response time to external events.

General-purpose operating systems — Windows, macOS, the many derivatives of Linux such as Ubuntu and Android, the BSD family, IBM's AIX — are optimised for running many applications efficiently on the same computer, and they talk to the user through some form of interactive shell.

This book is about general-purpose operating systems, built up from the bare machine, but the first few chapters are, in effect, embedded programming: one

program, in ROM, with no operating system at all. We look at the two families more closely, and at how a general-purpose computer gets from the power switch to the shell, in the chapter on general-purpose operating systems and booting, which also takes MS-DOS apart as a case study.

1.4 From batch processing to time-sharing, Unix and the microcomputer

The ideas in this book were invented between roughly 1955 and 1990. Many of the people who invented them explained their work on film, and I think the best way to meet an idea is from the person who had it. So I recommend four historic films, listed in Table 1.1. They take about two hours to watch, and they put the whole subject into perspective. This section tells the story they belong to, and says what to look out for in each.

Year	Film	Who and what
1962	*Digital Computer Techniques: Programming* (US Navy)	A training film on programming in machine code: flowcharts, instruction words, storage addresses.
1963	*Timesharing: A Solution to Computer Bottlenecks* (MIT)	Fernando Corbató explains and demonstrates time-sharing, and the supervisor program that makes it work.
1982	*UNIX: Making Computers Easier to Use* (AT&T Bell Laboratories)	Ken Thompson, Dennis Ritchie, Brian Kernighan and colleagues introduce the kernel, the shell, utilities, pipelines and the file system.
1991	BBC, history of computing, part 7: the graphical user interface and the microprocessor	Hobbyists, the first microcomputers, and the Xerox researchers who invented the graphical user interface.

Table 1.1: Four historic films on the ideas of this chapter. At the time of writing they are on YouTube: `youtube.com/watch?v=dFZecokdHLo`, `youtube.com/watch?v=Q07PhW5sCEk`, `youtube.com/watch?v=XvDZLjaCJuw` and `youtube.com/watch?v=jhvJtJg5g1s`.

1.4.1 Programs as numbers

The first computers were single-program, bare-metal machines. When a program ran, it had the entire computer to itself, and it was written in the computer's own language: numbers. The 1962 US Navy film is a good place to see what that meant. A program is a list of *instruction words*, each a number in which some digits say what operation to perform and other digits say where the data comes from and where the result goes. The words are placed in consecutive storage addresses, and the computer takes one after another in sequence, advancing automatically from each address to the next, unless an instruction tells it to jump elsewhere. Before writing the numbers, the programmer draws a *flowchart*, with rectangles for operations and diamonds for decisions.

It may or may not surprise you that this is still exactly how a computer works. The RISC-V processor we program in this book, one of the newest instruction set ar-

chitectures, designed in the 2010s, fetches binary instruction words from consecutive addresses in exactly the same way. What has changed is that we no longer write the numbers by hand: an *assembler* translates a human-readable form of each instruction into its number, and a compiler translates C into that form. Watch this film first if you have never programmed in assembly language; it introduces the ideas of the next chapter better than many textbooks.

1.4.2 Batch processing

The computers of the 1950s and 1960s filled a room. They were expensive to buy and expensive to run; an hour of computer time could cost a few hundred US dollars. Wasting it was not a good idea, and a program that had the machine to itself wasted most of it: the CPU is fast, and most of the time it was waiting — for a card reader to deliver data, for a printer to finish, for the programmer at the console to decide what to do next. Raising the *utilisation* of the CPU was the primary concern.

The first answer was *batch processing*. Collect the programs of, say, ten programmers, make them non-interactive, and run them one after another in a batch, rather like the commands in a shell script or a Makefile. Some time goes on loading each program, but the share of time spent calculating goes up. For a payroll program that calculated the salaries of ten thousand employees on the organisation's only computer, this was a good use of the equipment. For programmers it was painful. You punched your program onto cards, handed them in, and the next morning received a compilation error; you spent five minutes fixing it, handed the cards in again, and the next morning found a simple mistake in the arithmetic. One typing error cost a day.

Engineers then noticed that even in a batch the CPU spent much of its time feeding data to slow peripherals. Two changes followed. The first was hardware: a separate processor, the *input/output channel*, that takes data from the CPU quickly and passes it on to the devices at their own pace. Beside the central cabinet of an IBM System/360 or 370 mainframe stood cabinets of much the same size that did nothing but input and output. The second was software. A program that has started an input/output operation has nothing to do until it finishes, so there is no reason to let the CPU idle: put the program at the back of the queue and run the next one that is ready. This *multitasking batch processing* kept both the CPU and the peripherals busy nearly all the time, and the managers were delighted. The programmers were not: they still waited hours for their results, apart from a privileged few who had the computer to themselves at night.

1.4.3 Time-sharing

The next idea was to give each user a *terminal* — in effect an electric typewriter connected to the computer, often over a telephone line — and to let the computer switch quickly from one user's program to the next. Computers are fast and people are slow, so if the switching is quick enough, every user has the impression of having the computer to themselves. They can start programs when they like, think for as long as they need before typing the next input, and tear the printout off the typewriter for their report. A user who is doing nothing is simply skipped. This is *time-sharing*, and the key to it was the ability to *force* the processor to switch from one program to another, whether the program was ready to give it up or not.

The 1963 film shows time-sharing being explained by one of its inventors, Fernando Corbató, whose Compatible Time-Sharing System (CTSS) ran at MIT from

1961. He describes the problem — the bottleneck of a single expensive machine and a long queue of users — and demonstrates the system from a terminal. Listen for three things. The first is the *supervisor*, Corbató's name for the master program that is always present in memory, starts and stops the users' programs, mediates between them and the users' consoles, and organises the storage of their programs and data on disk. Today we call it the kernel. The second is his analogy for time-sharing: a chess master playing a large number of opponents at once, moving from board to board, while each opponent has only one game to think about. The third is the device the supervisor uses to make sure that no program keeps the processor too long. Corbató calls it an alarm clock; we will call it a timer interrupt, and build one in the chapters on interrupts and on preemptive multitasking. Corbató received the Turing Award in 1990 for this work and for Multics.

History: From Multics to the file cabinet

CTSS was followed by an ambitious successor, Multics, begun in 1964–65 at MIT's Project MAC together with General Electric and Bell Telephone Laboratories. Multics was meant to be a "computer utility" that would serve a whole community, as the electricity grid does. Among many other things, it gave us the hierarchical file system: in 1965 Robert Daley and Peter Neumann described named directories that contain files and further directories, the file-cabinet picture every operating system has used since. We read their paper in the chapter on file systems.

1.4.4 Unix

Bell Labs withdrew from Multics in 1969, but the people who had worked on it stayed, and one of them, Ken Thompson, started a hobby project: a small operating system for a PDP-7 minicomputer he had access to. Where Multics set out to do everything, the new system would do one thing and do it well, and its name, Unix, is a pun on Multics (it is usually credited to Brian Kernighan). Thompson was soon joined by Dennis Ritchie, and later by other former Multics people. Many great inventions start as hobby projects; do not underestimate yours.

Two things made Unix a success beyond Bell Labs. The first was C, the language Ritchie created and in which Unix was rewritten in 1973. Once a C compiler existed for a computer, Unix could be compiled for it, so it spread across the many different machines that research organisations owned. The second was the network. When the US Defense Advanced Research Projects Agency wanted the new Internet protocols to run on the many kinds of computer at its universities, it paid for TCP/IP to be built into the Berkeley version of Unix, and the Internet and Unix grew up together. Unix's programming interface was standardised by the IEEE in 1988 as POSIX, and today we distinguish systems descended directly from the Unix code, such as the BSDs, HP-UX and AIX, from clean re-implementations of its ideas, chiefly Linux.

The Unix interface is the masterpiece I mentioned at the start of the chapter. Its designers took the abstractions that had been developing over the previous fifteen years — files, directories, processes, devices — and realised them with a surprisingly small number of system calls that combine well. It was so successful that almost every operating system since provides much the same functionality in some form. The 1982 AT&T film lets you hear Thompson, Ritchie and Kernighan explain it themselves, and it introduces the vocabulary we still use: the *kernel*, which they describe as the operating system proper, controlling the machine and scheduling

the programs; the shell, the command interpreter that stands between the user and the system; the utilities, small programs that each do one task, such as editing or sorting a file; and *pipelines*, in which the output of one program goes directly into the input of the next. Look out for Kernighan building a spelling checker on the spot by joining a few such programs into a pipeline.

Notice also what the speakers say about people. From the start, Unix was meant not only as a good environment to program in but as a system around which a community could form; Ritchie wrote later that this had been their aim. An idea you keep in your head and never share is yours alone; it is the ability to involve others that makes an idea stick. Unix was written by essentially two people, with the help of colleagues, and it outlived almost everything around it. Thompson and Ritchie received the Turing Award for it in 1983.

1.4.5 The microcomputer and the graphical user interface

While Unix spread through universities, the microprocessor — a whole CPU on one chip, starting with Intel's 4004 in 1971 — made computers cheap enough for individuals. The people who most desperately wanted computers of their own in the 1970s were technology hobbyists, and the first microcomputers, such as the Altair 8800 of 1975, were kits for them. Small operating systems followed: CP/M for the hobbyist machines, and then MS-DOS for the IBM PC of 1981, which, together with its many clones, became the most widespread computer of the 1980s. Meanwhile at Xerox's Palo Alto Research Center, researchers asked what would make computers easy for everyone to use, and decided that the answer lay in skills we all already have: we are all experts at seeing and grabbing. Their Alto computer of 1973 had a mouse, overlapping windows and icons; Apple brought the idea to the mass market with the Macintosh in 1984. The 1991 BBC documentary tells this story with many of the people involved.

The early microcomputer operating systems were a step backwards in one sense. MS-DOS ran one program at a time and had no protection at all: any program could overwrite any part of memory, including the operating system. Over the next fifteen years microprocessors acquired the features mainframes had had since the 1960s — interrupt controllers, timers, privileged modes, memory management units — and microcomputer operating systems acquired multitasking and protection to match, in Windows NT, Linux (begun in 1991) and, on the Mac, Mac OS X. The same ideas were invented twice, once for the mainframe and once for the microcomputer, and they will be invented a third time in this book, for our simulated computer. Figure 1.2 summarises the story.

Try it

Watch the four films in Table 1.1, in order. For each one, write down at least one concept it introduces, the name of the person who introduces it, and two or three sentences explaining the concept in your own words. Then, for each concept, note which chapter of this book you expect to deal with it. Keep the notes: you will find yourself coming back to the films as the book goes on.

1.5 The kernel

In the old days it was called the supervisor; today we call it the *kernel*. Whatever the name, it is the one program of the operating system that is permanently resident in

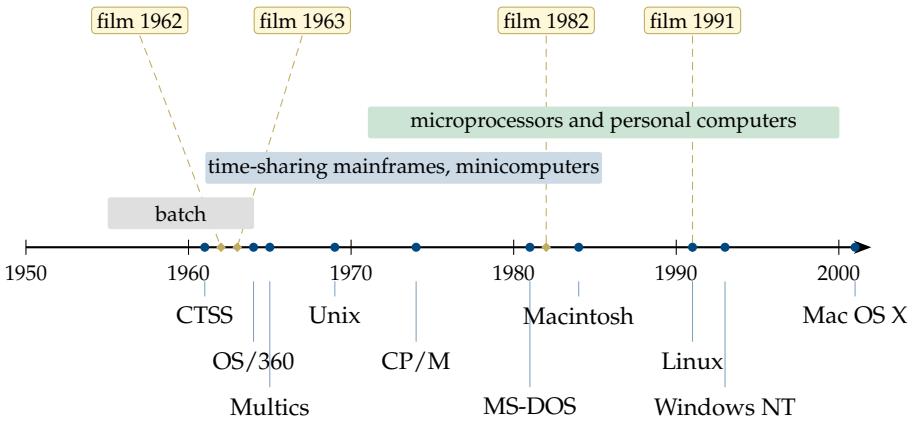


Figure 1.2: A rough timeline of the operating systems mentioned in this chapter, with the dates of the four films. The eras overlap: batch systems ran on for decades after time-sharing arrived, and time-sharing minicomputers coexisted with personal computers throughout the 1980s.

memory while the computer runs, and it is the part we will spend most of the book building and taking apart.

To see where it fits, consider the hardware a general-purpose computer must have to run an operating system at all (Figure 1.3). There is a CPU that can run code, perhaps with several cores. There is ROM holding an initial program, the *bootloader*, whose job is to load the operating system. There is a *console*, a display and keyboard through which the user interacts with the machine. There is a data storage device holding a file system. And there is RAM, into which programs are loaded from the file system to run.

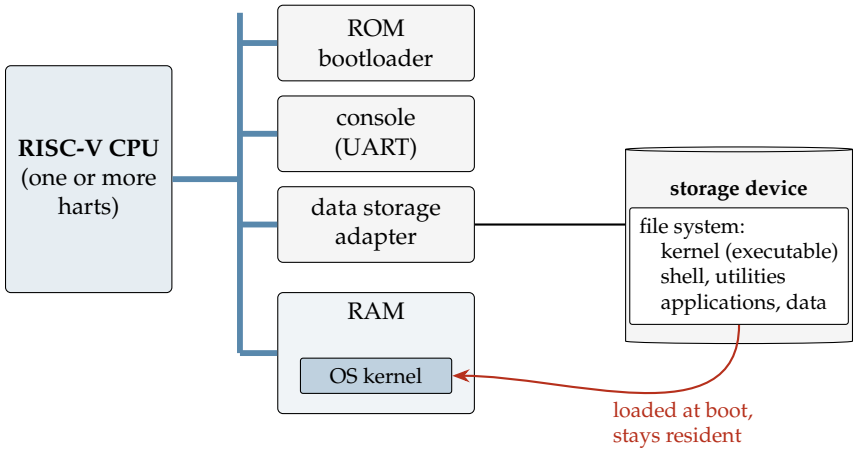


Figure 1.3: A general-purpose computer system. The bootloader in ROM loads the kernel from the file system into RAM, where it stays for as long as the computer runs; the kernel in turn loads programs from the file system into the free parts of RAM.

The operating system needs two components. One is the kernel: an executable program, loaded from the file system into RAM when the computer starts, which stays there until it is switched off. The other is the file system on the storage device, which contains the kernel itself, the utility programs, and whatever applications and data we add. While the computer runs, the kernel sits in memory and works with the file system: it loads applications into free parts of RAM and runs them; they in

turn read and write files and start other applications, through the kernel.

Once it is running, the kernel has a long list of responsibilities. It

- initialises the peripheral devices and ensures their proper operation;
- can launch programs contained in the file system;
- implements an interface of *system calls* through which running programs access the resources of the system;
- provides running programs with useful abstractions, such as files and folders;
- protects programs running at the same time from interfering with each other;
- provides means for running programs to interact with each other (interprocess communication);
- ensures that the resources of the system are used efficiently.

Two items on that list depend on the hardware in an essential way. To protect programs from one another, a running program — a process — must be kept away from the hardware and from other processes' memory, by a memory-protection mechanism or, most commonly, by *virtual memory*. When a process does need the hardware, it calls a special function of the kernel, and the kernel performs the operation on its behalf. That call cannot be an ordinary function call, because an ordinary call gives the caller no less power than the function it calls; it needs a hardware mechanism that switches the processor into a privileged mode at a fixed entry point of the kernel. On RISC-V it is the `ecall` instruction, and the chapter on system calls follows it all the way into the logOS kernel and back.

1.5.1 Monolithic kernels and microkernels

How much should the kernel contain? There are two basic approaches. In a *monolithic kernel* everything the operating system does is done inside the kernel: the file system, the drivers, the scheduler and the rest form a single large program, all of which must be correct, and it is almost impossible to write a large program without mistakes. Drivers are a particular worry, because they are written by the manufacturers of each device, teams who are not part of the operating system's own team and may not share its standards; an error in a driver running inside the kernel can bring the whole system down, or let an attacker take it over.

A *microkernel* squeezes the kernel to the minimum — creating and scheduling processes, managing memory, controlling access, and passing messages efficiently between processes — and moves everything else out into ordinary processes. A file system becomes a *file server*, a process that other processes send requests to; a disk driver becomes another process. If the file server crashes, the rest of the system survives, and a program that does not need files carries on. In the 1980s and 1990s this design was the height of fashion in operating systems research. The price is speed, since every request now travels as messages through the kernel. Linux and the traditional Unix kernels are monolithic, as is logOS; Windows NT and Apple's XNU kernel combine elements of both.

Key idea

The kernel is the part of the operating system that stays resident in memory, controls the hardware and offers everything else its services through system calls; the shell and the utilities are ordinary programs that use those services.

1.6 Code: the first thing the logOS kernel does

All of this is easier to believe when you see it. Here is the beginning of the `main()` function of the logOS kernel, which runs after the bootloader has copied the kernel from the file system into RAM at `0x00100000` and jumped to it.

```
83      printf("Welcome to logOS 2.0 Blackrock!\n\n");
84
85      /* Initialize subsystems */
86      console_dev_init();
87      proc_init();
88
89      /* Mount the filesystem */
90      printf("Mounting filesystem...\n");
91      result = fs_mount();
92      if (result != FS_OK) {
93          printf("ERROR: fs_mount failed with code %d\n", result);
94          return result;
95      }
96      printf("Filesystem mounted.\n\n");
97
98      /* Enable timer interrupts for preemptive scheduling */
99      timer_init();
100
101     printf("Starting shell\n");
102     printf("Type 'help' for available commands\n\n");
103
104     /* Run shell in a loop - restarts after each shell exit */
105     for (;;) {
106         int slot = proc_alloc();
```

Listing 1.1: The start of the logOS kernel (logOS Blackrock, `kernel/main.c`, lines 83–106).

Almost every item in the list of the kernel's responsibilities can be seen here in miniature. Line 86 registers the console driver, the kernel's interface to the text display and keyboard; until it has run, the kernel cannot print anything except through the bare device registers. Line 87 initialises the *process table*, the kernel's record of the programs it is running, each in its own fixed slot of memory. Line 91 *mounts* the file system: it reads the first block of the storage device, checks that it really holds a logOS file system, and makes its files and directories available. Line 99 starts the timer, Corbató's alarm clock, so that the kernel regains control of the processor at regular intervals whatever the running program is doing. Then, from line 105, the kernel enters an endless loop in which it loads the shell, `/bin/sh`, from the file system, runs it, and loads a fresh one whenever it exits; Chapter 10 walks through this loop in its simplest form. From then on the kernel runs only when something calls it: a system call from a program, or an interrupt from a device or the timer.

Notice what is *not* in the kernel. The shell that prints the prompt and reads your commands is an ordinary program in the file system, loaded like any other; so are `ls`, `cat` and `ps`. That the shell needs no special privileges is a sign of how much the system-call interface can do, and it is exactly the boundary drawn in Figure 1.1.

1.7 The simulated RISC-V computer and logOS

We will build our understanding on a computer that you can see all the way down to the gates. It is a RISC-V computer simulated in Logisim-evolution, the digital logic simulator, and it runs a small Unix-like operating system, logOS, that I wrote for teaching.

1.7.1 Why RISC-V

RISC-V is an *instruction set architecture*: a specification of the instructions, registers and behaviour that programmers can rely on, independent of any particular circuit that implements them. It was designed by academics at the University of California, Berkeley, and it is open and royalty-free: anyone may implement it in hardware without paying anyone anything. It is *modular*. A small base set of integer instructions, enough for a C compiler to target, is called RV32I in its 32-bit version, and optional extensions are indicated by letters after the name: M adds multiplication and division, Zicsr the instructions for control and status registers that an operating system needs. Our processor implements RV32IM with Zicsr. RISC-V is modern, clean and easy to build in hardware, and it is well documented; and because it is an industry standard, what you learn about it applies to real chips. The next chapter introduces its assembly language.

1.7.2 The machine

The computer is built in Logisim-evolution, UCD edition: a version of the open-source simulator, extended with a RISC-V processor component and other parts that a group of students helped me to design. You will have met Logisim if you have built digital circuits; here it lets us see hardware and software in one integrated environment. We can watch a program run instruction by instruction, open the processor and look at its registers, and wire in new devices of our own. Its memory map is shown in Table 1.2.

Device	Address	Use
ROM	`0x00000000`	Program run after reset: a bare-metal program in Part I, the logOS bootloader later.
RAM	`0x00100000`	Data and stack; under logOS, the kernel and the process slots.
Block storage	`0x00200000`	Registers of a simulated disk of 512-byte blocks, which copies blocks to and from RAM by direct memory access.
Timer	`0x02004000`, `0x0200BFF8`	`mtimecmp` and `mtime`, for timer interrupts.
Keyboard	`0xFFFF0004`, `0xFFFF0008`	Receiver control and data registers.
TTY display	`0xFFFF000C`	Writing a character code prints it.

Table 1.2: The main devices of the simulated computer and their addresses. Some programs add others, such as the 32×32 VGA display at `0xFFFF8000` used for Pong.

The machine has one core. In its basic form, used for most of the book, it also has no memory-management unit, so every address is a physical address and there is

no hardware memory protection: any instruction can access any address. That is a limitation, but a deliberate one, at least to begin with. It is how the microcomputers of the 1970s and early 1980s worked, and it lets us meet each mechanism in its simplest form before we add the hardware that makes it safe: for the chapter on virtual memory, the processor gains an Sv32 memory-management unit and separate machine, supervisor and user modes. When we need features the Logisim machine does not have, or more speed than a gate-level simulation can give, we turn to emulators, as described below.

1.7.3 logOS

logOS is a teaching operating system inspired by Unix. It provides a subset of the services and system calls that Unix kernels traditionally offer — processes, file descriptors, a file system with directories, device files and a shell, and from version 2.0 pipes — and it is small enough, a few thousand lines of C and assembly, for you to read every line. The kernel is organised in layers: block input/output at the bottom, then i-nodes, directories, files and the file system; device drivers; trap handling; processes; system calls; and the loader that brings executable files from the file system into memory. It comes in versions of growing scope, each with its source code and a textbook that explains it (Table 1.3).

Version	Hardware	Main features
1.0 Belfield	RV32IM with Zicsr, machine mode only; no timer needed	Single-tasking: a program runs another with `spawn` and waits for it to finish, which makes it functionally closer to MS-DOS than to Unix. File system, device files, loading of position-independent executables, environment variables, shell, basic utilities.
2.0 Blackrock	As 1.0, plus the timer	Preemptive multitasking with a round-robin scheduler; `fork`, `exec` and `wait`; pipes and pipelines; input/output redirection; shell scripts; shared memory and semaphores.
3.0 Glendalough	As 2.0, plus an Sv32 memory-management unit and machine, supervisor and user modes	Virtual memory: each process has its own address space, and the hardware enforces the isolation between processes and the kernel.

Table 1.3: The versions of logOS.

The versions follow the growth of the book. Belfield, the simplest, shows the essentials — how a system call enters the kernel, how a path name is turned into data on the disk, how the shell runs a program — without any concurrency at all. Blackrock adds what Part III is about: processes that run at the same time, and the means for them to communicate. Glendalough adds the hardware protection of Part IV. In all three, physical memory above the kernel is divided into fixed slots, one per process, starting at `0x00110000`: eight slots of 64 KB in Belfield, eight of 32 KB in Blackrock and Glendalough. In Glendalough, however, every process sees the same virtual addresses, whichever slot it occupies.

Try it

Boot logOS Blackrock in Logisim.

1. In a terminal, `cd` into your `logos` folder (the `2.x-blackrock` branch of the repository, set up as Appendix C describes) and type `make all`. This builds the bootloader ROM image `boot-rom.txt`, the kernel, the user programs, and a file-system image `block_storage.bin` that contains them all.
2. Open `computer.circ` in Logisim-evolution (UCD edition) and check that the ROM holds the bootloader and the block device holds `block_storage.bin` (see the logOS `README.md`). Enable auto-tick and watch the terminal: the bootloader reports `Bootloader starting...`, `BOOT: Filesystem found` and `BOOT: Jumping to kernel at 0x100000`, and then the kernel prints the messages of Listing 1.1.
3. When the prompt appears, click the keyboard and type `ls`, then `cd /bin` and `ls` again. Each command is a program loaded from the simulated disk, so be patient.
4. Now quit and run the same circuit from a terminal with `java -jar logisim-evolution-ucd-5.0.7-all.jar computer.circ --tty raw`. Logisim no longer draws the circuit; the terminal is connected directly to the simulated keyboard and display, and the system is much faster. Try `ps`, `cat /etc/rc` and `help`. Which of the commands you typed were programs in `/bin`, and which were handled by the shell itself?

1.8 How to use this book

After this introduction, the book climbs from the hardware upwards in four parts.

Part I: The machine (Chapters 2–4)
We program the simulated computer with no operating system at all: RISC-V assembly language and memory-mapped input/output; the tools that turn source code into a running program — assembler, linker, linker scripts, `make`; and C on bare metal, with pointers to device registers, the C runtime and the stack, and direct memory access to the block storage device.

Part II: The operating system from the outside, and logOS Belfield 1.0 (Chapters 5–10)
General-purpose operating systems and how they boot, with MS-DOS as a case study; file systems; system calls and the Unix programming interface; the shell; shell scripting. Each topic is first seen as a programmer sees it, and then in the code of logOS *Belfield 1.0*, which Chapter 10 assembles into a working whole.

Part III: Doing many things at once, and logOS Blackrock 2.0 (Chapters 11–16)
Cooperative multitasking (Chapter 11), interrupts (Chapter 12) and preemptive multitasking lead to *Blackrock 2.0*; then pipes and scheduling algorithms.

Part IV: Sharing the machine safely, and logOS Glendalough 3.0 (Chapters 17–22)
Synchronisation; semaphores, deadlock and monitors; virtual memory, leading to *Glendalough 3.0*; multiprocessors; protection and access control.

Each part therefore ends with a new version of the same system, built from the ideas of that part. Code in each chapter comes from the version you have reached. Appendices give a RISC-V quick reference (Appendix A), the full description of the

simulated computer (Appendix B), instructions for setting up the tools (Appendix C) and a guide to the logOS test suite (Appendix D).

Most chapters have the same shape. They open with a concrete problem on our machine, build the concepts needed to solve it, and then walk through real code in sections marked *Code:*, quoted exactly from the book's repository or from logOS, with line numbers that match the files. Short snippets that are *not* from that code are labelled as illustrations. A *Real world* section then compares our approach with Linux, Unix, Windows, embedded systems or historic machines, and each chapter ends with exercises, from questions of understanding through tracing and modifying code to design problems.

Three kinds of box appear in the text:

- **Try it** boxes are hands-on steps to carry out in Logisim, RARS or QEMU. Do them: the point of a simulated computer is that you can watch everything happen.
- **History** boxes are historical asides about where an idea came from and who had it.
- **Key idea** boxes hold the one or two sentences of a chapter that are most worth remembering.

1.8.1 Tools

Everything in the book can be done with free software, on your own computer: on a Mac in its Terminal, on Linux, or on Windows in Ubuntu under the Windows Subsystem for Linux (WSL).

- **Logisim-evolution, UCD edition**, the simulator of our computer. It runs anywhere Java does, with its graphical interface or, for speed, from the command line with the terminal attached to the simulated console.

- **RARS**, the RISC-V Assembler and Runtime Simulator. Logisim is not designed for developing software: to see a register you have to click several levels down into the processor circuit. RARS is a small graphical environment in which you type an assembly program, assemble it and run it on the spot, with the registers and memory in view. We use a modified edition whose devices and control registers resemble those of our Logisim machine, so that small programs behave the same in both.

- **The GNU toolchain for RISC-V**: the C compiler, assembler and linker, called (`riscv64-elf-gcc`, `riscv64-elf-ld` and their companions), `make`, and the GNU debugger `gdb`.

- **QEMU**, a fast emulator of complete computers, including RISC-V machines with several cores and a memory-management unit. We use it where the Logisim machine is too small or too slow, above all for multiprocessors.

- **A Unix command line**: the Terminal of macOS, a Linux system, or Ubuntu under WSL on Windows. It is where all of the above run, and it doubles as the Unix system on which we try out the shell and system calls. The sessions in the book were recorded on Ubuntu Linux, which is also what WSL provides; on a Mac the same commands work, with small differences here and there.

Appendix C explains how to install each of them; a script that comes with logOS does most of the work.

You can read the book on its own, but it was written to be used: keep a terminal open as you read, build each program, and change it. When something does not behave as the text says it should, find out why. That is how operating systems are learnt.

1.9 Real world

Operating systems today. Almost every general-purpose computer now runs one of three families of system. Linux runs the great majority of servers and supercomputers and, inside Android, most of the world's phones. Apple's macOS and iOS run on Darwin, whose XNU kernel combines the Mach microkernel from Carnegie Mellon with large parts of BSD Unix. Windows is built on the Windows NT kernel, designed in the early 1990s by a team led by Dave Cutler that came from Digital Equipment Corporation's VMS rather than from Unix. All three provide processes, threads, hierarchical file systems, virtual memory and a system-call interface, and the first two are Unix in all but name.

Embedded and real-time systems. Far more computers run no general-purpose operating system at all. Small microcontrollers often run a single bare-metal program, like the ones in Part I, or a real-time operating system supplied as a library, such as FreeRTOS or Zephyr, which provides tasks, timers, semaphores and message queues and little else. Between the two families sits *embedded Linux*, which runs in routers, televisions and cars wherever the hardware is large enough.

RISC-V in practice. RISC-V cores are now common inside microcontrollers and as controllers within larger chips, and there are RISC-V boards and laptops that run Linux. The Linux kernel, FreeBSD, FreeRTOS and Zephyr all support RISC-V. The privileged architecture that our operating system will rely on — machine mode, interrupts and the timer, and later the Sv32 and Sv39 page tables — is the same in our simulated machine as in these real chips.

Teaching operating systems. logOS belongs to a long tradition of small operating systems written to be read. Andrew Tanenbaum wrote MINIX in 1987 to accompany his textbook, and it was MINIX that a Helsinki student, Linus Torvalds, used and set out to improve upon when he began Linux in 1991. At MIT, xv6 re-implements Sixth Edition Unix in modern C, now for RISC-V. logOS is smaller still, and is designed for a machine you can inspect down to the gates.

1.10 Exercises

1. *A definition.* Write your own definition of an operating system in no more than three sentences. Then decide, with reasons, which of the following you would count as part of the operating system of a Linux or Windows computer: the device driver for the graphics card; the shell; a text editor installed with the system; the web browser; the bootloader in ROM or firmware; the C library. Where does your answer agree with the boundary drawn in Figure 1.1, and where not?

2. *Embedded or general-purpose?* Classify each of these as an embedded or a general-purpose computer system, and say what kind of operating system, if any, you would expect it to run: a washing-machine controller; a smartphone; a car's

engine controller; a smart television; a Mars rover; a Raspberry Pi running Linux that does nothing but water a greenhouse. For which of them is the distinction least clear, and why? What changes when a device starts receiving software updates over the network?

3. *Four films.* Go back to your notes on the four films of Table 1.1. Which of the films' ideas can you find in the computer you are using now, and where?

4. *The value of a batch.* A program spends 2 minutes calculating and 8 minutes waiting for its card reader and printer. (a) If programs of this kind run one at a time, what fraction of the time is the CPU busy? (b) With input/output channels and multitasking batch processing, how many such programs would have to be in memory at once to keep the CPU busy all the time, assuming the channels can serve them all in parallel? (c) Why did neither arrangement satisfy the programmers, and what did time-sharing change for them?

5. *The alarm clock.* Corbató's supervisor had an alarm clock to stop programs that ran too long. What could one user's program do to all the others on a time-sharing system without it? Relate your answer to the Pong programs of Chapter 11, in which each task decides for itself when to give up the processor. Which line of Listing 1.1 sets up the logOS equivalent of the alarm clock?

6. *What the kernel needs from the hardware.* For each of the seven responsibilities of the kernel listed in Section 1.5, say whether it needs special support from the hardware, and if so what kind (for example: a storage device, an interrupt, a timer, a privileged mode, memory protection). Which of them can logOS provide on a machine with no memory-management unit, and which only by convention? Keep your answer and compare it with what you know at the end of the book.

7. *Small programs, combined.* In your terminal, take any text file and use a pipeline of standard utilities to print the ten words that occur in it most often, with their counts. (Hint: `tr`, `sort`, `uniq` and `head` are enough.) How many programs run at once while your pipeline works? Compare your pipeline with the spelling checker in the 1982 Unix film. What does this way of building software require of the operating system?

8. *Monolithic or micro?* A smartphone manufacturer ships drivers for its camera and modem that are written by other companies. Argue the case for running those drivers inside a monolithic kernel, and the case for running them as separate processes on a microkernel. Which arguments concern reliability, which security and which speed? Which would you choose for a heart pacemaker, and which for a games console?

Part III

Doing many things at once: logOS Blackrock 2.0

Chapter 11

Cooperative multitasking

A Mars rover has to steer its wheels, watch the ground ahead for rocks and crevices, test its own hardware for faults, talk to Earth, collect and store scientific data, and keep its batteries from running flat — all at more or less the same time. A fridge controller has to hold the temperature steady while it answers the buttons on the door. A computer game has to read the keyboard while it moves every other character on the screen. Yet a CPU core does exactly one thing: it fetches one instruction, executes it, and fetches the next. With a single core there is only ever one stream of instructions. So how can we possibly program a machine to do several things at once?

Long before an operating system enters the picture, this is a question about how we arrange our own program, and that is where this chapter starts. I take one small program — a dot bouncing around a 32×32 pixel screen whose position you can change by typing coordinates — and write it three ways on the bare-metal RISC-V computer: by letting one activity block the other, by rewriting the input handling as a finite-state machine, and by turning the two activities into *coroutines* that take turns on the CPU. The third way introduces the two ideas the rest of Part III is built on: the *execution context* of a running program, and the *context switch* that saves one context and restores another.

11.1 Programs that do several things at once

The rover is an extreme case (and, to be fair to its designers, it has several computers on board, not one), but ordinary programs face the same problem. A business application has its core computation to do — creating users, calculating the payroll — and at the same time it must handle input from the keyboard, query a local database and talk to a server across the Internet, without any of these activities freezing the others. Your laptop moves the mouse pointer smoothly while it compiles a large program in the background. Each of these programs consists of several *tasks*, each with its own sequence of steps, and the tasks must make progress together.

With more than one processor core we could give each task a core of its own; that is *multiprocessing*, the subject of Chapter 21. But there are always more tasks than cores, and the small computer we have been programming has exactly one. The alternative is to switch a single core rapidly between the tasks, so that each makes a little progress in turn. This is *multitasking*. In this chapter the switching is done by the program itself, at points the programmer chooses; in the next two chapters we will make the hardware force it.

11.2 My Pong "game"

To have something concrete to work on, I wrote a small program that I call Pong, although it is not really a game of ping-pong. A white dot moves across a graphical display, bouncing off its borders. Next to the display is a text terminal and a keyboard. When the program starts, it prints

```
input new coords as X,Y<Enter>
```

and if you type, say, `15,10` and press Enter, the dot jumps to column 15, row 10 and carries on bouncing from there. Two things therefore happen at once: the program moves the dot, and it reads and interprets what you type, key by key.

11.2.1 The machine

The program runs on the same simulated RISC-V computer we used in Part I (Table 2.9): the program in ROM from address `0x00000000`, RAM at `0x00100000` (of which the Pong linker script uses the first 64 KB), and the memory-mapped terminal and keyboard at the top of the address space. For Pong I added one new device, a VGA-style colour display, which occupies the addresses `0xFFFF8000` to `0xFFFF8FFF`.

The display is small: 32 pixels wide and 32 high, so $32 \times 32 = 2^{10} = 1024$ pixels. Each pixel is a separate 32-bit word in memory, holding its colour in the familiar web format `0xRRGGBB`: `0xFFFFFF` is white, `0x000000` black. The pixels are stored row by row, so the pixel in column x of row y lives at

$$\texttt{0xFFFF8000} + 4\,(32y + x).$$

The origin $(0, 0)$ is the top-left corner, and y grows downwards.

Two small header files, `console.h` and `vga.h`, give the device addresses names. They are identical in all three versions of the program, and we have met them already, in Listing 4.6: `TTY` is the terminal, `RCR` and `RDR` are the keyboard's status and data registers, and `VGA` is the first pixel of the display, with `VGA_WIDTH` and `VGA_HEIGHT` both defined as 32. Each name is a `volatile` pointer to a fixed address (Section 4.3.1), so `*RCR` reads the keyboard status and `*TTY = c` prints a character. Because `VGA` is a pointer to `unsigned int`, we can treat the display as an array of 1024 words, and plotting a dot becomes a one-line macro:

```
22 #define VGA_PLOT(X, Y, COLOUR) ((VGA)[((Y) * (VGA_WIDTH)) + (X)] = (COLOUR))
23
24 /* Colour of the bouncing dot 0xRRGGBB */
25 #define DOT_COLOUR (0xFFFFFF)
26
27 /* print string to TTY */
28 void printstr(char *str)
29 {
30    while (*str)
31       *TTY = *str++;
32 }
```

Listing 11.1: Plotting a pixel (`main.c`, all three versions).

The index `Y*VGA_WIDTH + X` is the formula above without the factor of four, which C pointer arithmetic supplies (Section 4.4). The parameters are wrapped in brackets so that a call such as `VGA_PLOT(x+1, y, c)` expands to `(y)*(32)+(x+1)` rather than something with the wrong operator precedence. `printstr` copies a string to the terminal one character at a time until it reaches the terminating zero.

11.2.2 Two algorithms

The program has two jobs. The first is to bounce the dot. The dot is described by its position (x, y) and by its velocity (dx, dy), where each of dx and dy is either $+1$ or -1. Initially both have magnitude one, so the dot moves diagonally. The algorithm is a loop:

1. Calculate the new position by adding the speeds: `new_x = x+dx`, `new_y = y+dy`.
2. If the new position is outside the screen, bounce: change the sign of dx and/or dy, and recalculate the new position.
3. Erase the dot in the old position.
4. Plot the dot in the new position.
5. Make the new position the old one for the next iteration: `x=new_x`, `y=new_y`.

Started at $(10, 10)$ moving up and to the right, the dot traces the closed diamond shown in Figure 11.1, touching each border in turn.

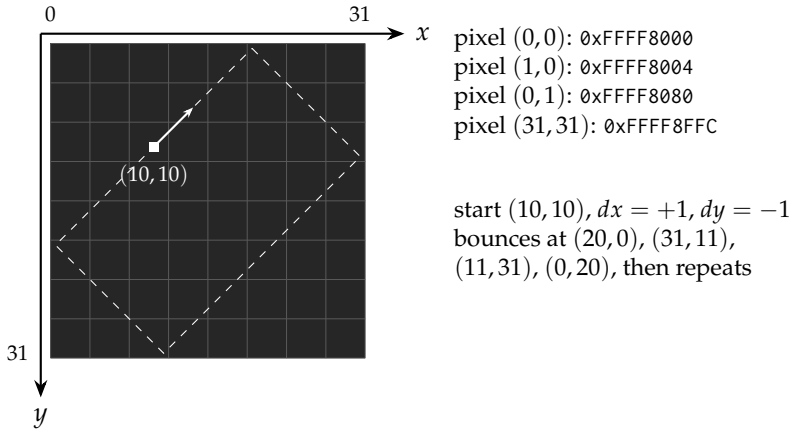


Figure 11.1: The 32×32 VGA display, the addresses of some of its pixels, and the path of the bouncing dot from its starting position.

The second job is to process what the user types. In outline:

- If the user pressed some key on the keyboard,
  1. read the new x coordinate of the dot;
  2. read the new y coordinate of the dot;
  3. if the entered coordinates are outside the screen borders, print an error message; otherwise erase the dot in its current position, plot it in the new position, and assign the new values to x and y.

Reading a coordinate is itself a loop, and the first version of the program does it with an ordinary function, `readint`:

```
34 /* read integer from keyboard */
35 int readint()
36 {
37    int res, key;
38    for (res = 0;;)
39    {
40       while (*RCR == 0)
41          ;
42       key = *RDR;
43       *TTY = key;
44       if ((key < '0') || (key > '9'))
45          break;
46       res = res * 10 + (key - '0');
47    }
48    return res;
49 }
50
51 /* returns 1 if a key had been pressed on the keyboard, or 0 otherwise */
52 int key_pressed()
53 {
54    return *RCR;
55 }
```

Listing 11.2: Reading a number from the keyboard (`blocking/main.c`).

The `while` loop on line 40 waits until a key is available. Line 42 takes the key out of the keyboard buffer and line 43 echoes it to the terminal, so that you can see what you are typing. (Writing the two steps separately is deliberate: `*TTY = *RDR` would print the key but lose it.) A digit is folded into the result in the usual way — multiply what we have by ten, add the value of the new digit — and the first non-digit ends the number. The comma between the coordinates and the Enter after them are therefore not special: any non-digit will do, so `5x5!` works just as well as `5,5`.

11.2.3 Why the two loops do not fit together

Each algorithm on its own is easy to write. The bouncing is one endless loop. The input processing is a loop around two calls of `readint`, each of which contains a loop of its own, which contains yet another loop waiting for a key. The difficulty is that we want both to run at the same time on one CPU, and while the CPU is executing the waiting loop on lines 40–41, it is doing nothing else. Nothing in that loop moves the dot.

An obvious objection is that the two activities change the same data: the bouncer updates x and y at every step, and the input code overwrites them. That is a real concern, and it will become a serious one when the switching between tasks is no longer under our control (Chapter 17 is devoted to it). But here it is not the main obstacle. The main obstacle is that one task *waits*, and while a single CPU waits in one task, it cannot work on the other.

There are three ways out, and the rest of the chapter takes them in turn:

1. Insert input processing into the dot-bouncing loop, and stop (*block*) the bouncing while the user types new coordinates.
2. Make input processing non-blocking by rewriting it as a software finite-state machine.
3. Define dot bouncing and input processing as coroutines.

I wrote one program for each; they are in the `blocking`, `fsm` and `coroutines` folders of the book's repository, and each builds with `make`.

Try it

Build and run the three Pong programs.

1. In a terminal, `cd` into `pong/blocking` and type `make`. Besides the ELF executable, the disassembly (`.asm`) and the linker map (`.map`), the Makefile produces `pong-blocking.bin.txt`: the ROM image in the text format Logisim reads.
2. Open `pong.circ` in Logisim-evolution (UCD edition). Right-click the ROM, choose *Load Image...* and select the `.bin.txt` file.
3. Reset the simulation and start the clock. At full speed the dot is a blur; lower the auto-tick frequency to a few hundred hertz (I use 256 Hz) to watch it move.
4. Click on the keyboard component with the poke (hand) tool and type `5,5` followed by Enter. Watch the terminal and the display while you type.
5. Repeat with the programs in `pong/fsm` and `pong/coroutines`. Which versions keep the dot moving while you type? Which runs fastest when you are not typing, and which when you are?

11.3 Approach 1: blocking

The first idea is the obvious one: do one iteration of the bouncing loop, then one iteration of the input loop, and repeat. If no key has been pressed, the input part has nothing to do, so it returns at once and the dot moves on. This is how a typical game loop is written, and for a game where each key needs only an instant reaction — move the player one step to the left — it works very well.

Our input is not like that. Entering a coordinate is a *process* that unfolds over many keystrokes. Once the user starts typing, the program has to keep reading digits until the end of the first number, then read the second number, and only then move the dot. If we simply call the input code from the game loop, the moment a key is pressed we enter `readint` and stay there until both numbers have been typed. This becomes step 6 of the bouncing algorithm: *if some key was pressed, process user input*. In `blocking/main.c` it looks like this:

```
84    for (;;)
85    {
86       new_x = x + dx;
87       new_y = y + dy;
88
89       if ((new_x < 0) || (new_x >= VGA_WIDTH))
90       {
91          dx = -dx;       // bounce along X
92          new_x = x + dx; // recalculate
93       }
94
95       if ((new_y < 0) || (new_y >= VGA_HEIGHT))
96       {
97          dy = -dy;       // bounce along y
98          new_y = y + dy; // reclaculate
99       }
100
101      VGA_PLOT(x, y, 0x000000);           // erase the dot by making it black
102      VGA_PLOT(new_x, new_y, DOT_COLOUR); // paint it in new coordinates
103
104      x = new_x;
105      y = new_y; // remember new coordinates
106
107      /* check keyboard and read new coordinates if the user typed something */
108      if (key_pressed())
109      {
110         new_x = readint();
111         new_y = readint();
112         if ((new_x >= VGA_WIDTH) || (new_y >= VGA_HEIGHT))
113         {
114            printstr("New coords out of bounds!\n");
115         }
116         else
117         {
118            VGA_PLOT(x, y, 0x000000);           // erase the dot by making it
     black
119            VGA_PLOT(new_x, new_y, DOT_COLOUR); // paint it in new coordinates
120            x = new_x;
121            y = new_y;
122         }
123      }
124   }
```

Listing 11.3: The game loop of the blocking version (`blocking/main.c`).

Lines 86–105 are the five steps of the bouncing algorithm, almost word for word. Lines 108–123 are the input algorithm, also almost word for word. Notice that the bounds check on line 112 tests only the upper limits: `readint` cannot return a negative number, because a minus sign is a non-digit and simply ends the number.

The variables `x` and `y` are globals, deliberately left uninitialised in their declarations and assigned at the start of `main`, because on our computer an initialised global would end up in ROM (Section 4.8). The comments in the source shout `(UNINITIALIZED!)` as a reminder.

Load the program into Logisim and it works as intended. The dot bounces, and the program checks the keyboard once per step. As soon as I type `5`, the dot stops. I type `,5` and Enter; the dot reappears at $(5,5)$ and resumes bouncing. While I am

typing, nothing moves.

We could, of course, turn this bug into a feature. Give the program to a salesperson and they will explain that it is better for the user's concentration: you either watch the dot bounce, like watching paint dry, or you type your coordinates, but you are never distracted by doing both. In many applications this really is acceptable — a program that stops what it is doing while you fill in a dialogue box is perfectly normal — but not in all. A rover cannot stop steering while it listens to Earth.

To sum up the blocking approach:

Pros: It is simple to implement, and it uses the CPU efficiently: no instructions are spent on anything except the two algorithms themselves.

Cons: One of the tasks stops while the other one progresses.

11.4 Approach 2: a software finite-state machine

How can we keep the dot moving while the user types? The direct answer is this: do not merely check for a key at the top of the game loop, but interleave the bouncing with the reading of the individual digits. We should be able to deal with one keystroke, go and move the dot, come back and deal with the next keystroke, and so on. The input algorithm must never sit and wait.

The tool for this comes from digital design. Any algorithm can be described as a *finite-state machine*: a system that, at every clock tick, looks at its current state and its inputs, produces some outputs, and moves to a new state. A flowchart with loops can be turned into a state machine by naming the points where it waits and listing what it does at each of them. Finite-state machines are natural in hardware, where a register holds the state and combinational logic computes the next one, but they are equally easy to build in software: a variable holds the state, and a `switch` statement inside a loop decides what to do in each state. If you have written event-driven code — in JavaScript, say, or in a game engine where a big `switch` is called a few dozen times a second — you have already programmed this way.

11.4.1 Finding the states

Look again at the input algorithm. It has three distinct stages:

1. waiting for the user to start typing;
2. reading the new value of x;
3. reading the new value of y.

In each stage a typed character is treated differently, just as a state machine reacts differently to the same input in different states. In the first stage, any key is a signal to start. In the second, a digit is added to x and a non-digit means "x is finished". In the third, a digit is added to y and a non-digit means "the coordinates are complete: check them and move the dot". Figure 11.2 draws this as a state machine. Each arrow is labelled *condition / action*, and "–" means "no action".

The machine takes one step every time the game loop goes round. In most steps no key has been pressed, and the machine stays where it is and does nothing, so the loop goes straight back to moving the dot. When a key *is* waiting, the machine consumes exactly one character, does a little work, perhaps changes state, and again returns to the game loop. Nothing ever waits.

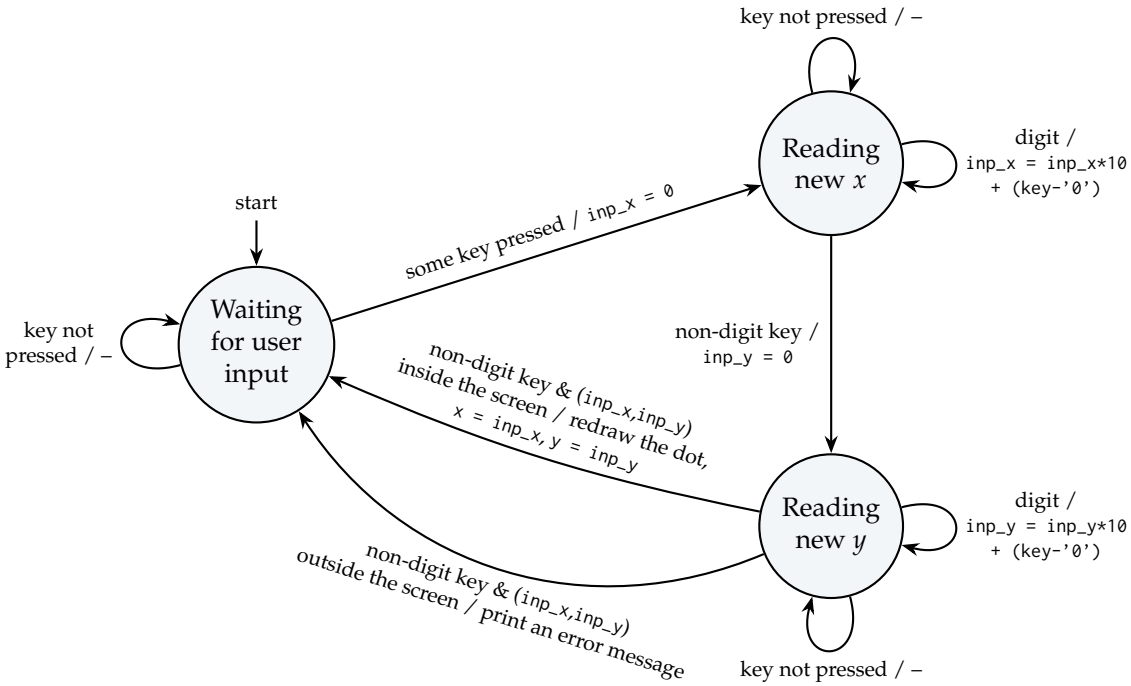


Figure 11.2: User-input processing as a finite-state machine. Each transition is labelled *condition / action*. The machine takes one step each time round the game loop.

11.4.2 From flowchart to switch

In the blocking version, the structure was three loops deep: the game loop, the loop inside each call of `readint`, and the waiting loop inside that. To turn it into a state machine we turn the loops inside out. All the inner loops disappear; the only loop left is the game loop, and in its body a `switch` on the current state does the work of one step. Here is the state, as an enumeration and a global variable:

```
40 /* Input Finite State Machine */
41 enum
42 {
43    FSM_WAITING_FOR_INPUT,
44    FSM_READING_X,
45    FSM_READING_Y
46 };
47
48 /* global (UNINITIALIZED!) variable for the state of input FSM */
49 int input_state;
```

Listing 11.4: The states of the input machine (`fsm/main.c`).

`main` sets `input_state = FSM_WAITING_FOR_INPUT` before the game loop starts, and declares two more local variables, `inp_x` and `inp_y`, to accumulate the digits as they arrive. The bouncing part of the loop is exactly as before (lines 84–105). The `if (key_pressed())` block at the end of the loop is replaced by the state machine:

```
107       /* check keyboard and read new coordinates if the user typed something */
108       switch (input_state)
109       {
110          int key;
```

```
111
112       case FSM_WAITING_FOR_INPUT:
113          if (*RCR == 0)
114             break;
115          input_state = FSM_READING_X;
116          inp_x = 0;
117          // fall through to process first digit of inp_x
118
119       case FSM_READING_X:
120          if (*RCR == 0)
121             break;
122          key = *RDR;
123          *TTY = key;
124          if ((key >= '0') && (key <= '9'))
125          {
126             inp_x = inp_x * 10 + (key - '0');
127             break;
128          }
129          else
130          {
131             input_state = FSM_READING_Y;
132             inp_y = 0;
133             // fall through to process first digit of inp_y
134          }
135
136       case FSM_READING_Y:
137          if (*RCR == 0)
138             break;
139          key = *RDR;
140          *TTY = key;
141          if ((key >= '0') && (key <= '9'))
142          {
143             inp_y = inp_y * 10 + (key - '0');
144             break;
145          }
146          else
147          {
148             input_state = FSM_WAITING_FOR_INPUT;
149             if ((inp_x >= VGA_WIDTH) || (inp_y >= VGA_HEIGHT))
150             {
151                printstr("New coords out of bounds!\n");
152             }
153             else
154             {
155                VGA_PLOT(x, y, 0x000000);           // erase the dot by making it
    black
156                VGA_PLOT(inp_x, inp_y, DOT_COLOUR); // paint it in new coordinates
157                x = inp_x;
158                y = inp_y;
159             }
160          }
161       }
```

Listing 11.5: The input state machine at the end of the game loop (`fsm/main.c`).

Each `case` is one state of Figure 11.2, and each `break` is the end of a step: control leaves the `switch`, reaches the end of the game loop and goes round to move the dot again.

Lines 112–117: waiting.
If there is no key, `break` — the self-loop "key not pressed". Otherwise we move to the reading-x state and clear the accumulator. Notice that we have *not* read the key: this state only notices that typing has begun. There is no `break` at the end of this case, so execution *falls through* into the next case and the same key is processed at once as the first character of x. Fall-through is usually a bug in C, which is why the comment on line 117 says it is intended.

Lines 119–134: reading x.
The test on line 120 is the second "key not pressed" self-loop; a key is certainly waiting when we fall in from the previous state, but not when we arrive here at the start of a later step. Lines 122–123 consume the key and echo it, exactly as `readint` did. A digit is accumulated, and the `break` on line 127 ends the step. A non-digit moves the machine to the reading-y state and falls through again.

Lines 136–160: reading y.
Arriving by fall-through, the test on line 137 usually finds the keyboard empty (we have just consumed the comma), so the step ends there. In later steps digits are accumulated into `inp_y`. The final non-digit sends the machine back to waiting and performs the action of whichever of the two return transitions applies.

Line 110.
The local variable `key` is declared inside the `switch` block, before the first label. That is legal C, but a declaration there is never "executed", so it must not have an initialiser.

Build this version, load it into Logisim, and type while the dot is moving. The dot keeps bouncing while you type; each character appears on the terminal as soon as you press the key; and when you press Enter the dot jumps to its new position. If you type coordinates outside the screen, the error message appears and the dot carries on regardless.

11.4.3 What the state machine costs

Compare the two versions of the input code. In the blocking version, the information "where are we in the input algorithm?" was held implicitly by the processor: it was in the program counter (which loop we were executing), on the stack (which of the two calls of `readint` we were in, and where it would return to), and in local variables of `readint`. To make the algorithm non-blocking we had to give all of that up, because every step must return to the game loop and every return destroys the stack frame of the function that returns. So we moved the information into variables that survive between steps: `input_state` records what the program counter used to record, and `inp_x` and `inp_y` take over from `res`. This observation is the key to the third approach, so remember it.

The rewrite also cost us readability. Can you tell at a glance what algorithm the `switch` statement of Listing 11.5 follows? With effort you can work it out, but the short description we started from has dissolved into a tangle of `case`s, `break`s and fall-throughs — and this is about the simplest input algorithm imaginable. Real games and simulations are programmed in this style, and plenty of event-driven code is too. I dislike it, because state machines written this way are hard to understand and hard to maintain.

Pros: Efficient use of the CPU, and both tasks make progress.

Cons: A major rewrite of one or more algorithms is needed to make them non-blocking, and FSM "algorithms" are hard to understand and maintain.

That said, when the CPU is very weak and nothing better is available, this is what programmers resort to, and it is still how much small embedded software is written.

11.5 Approach 3: coroutines

Is there a way to keep both algorithms in their original, readable form — one with its single loop, the other with its nest of loops inside `readint` — and still have them run together? There is, and it rests on the observation we made about the state machine: it had to keep track of its own place in the algorithm because there is only one program counter and one stack, and the game loop needed them. So let us give each task its own.

The idea is this:

- Dot bouncing and user-input processing are treated as semi-independent *tasks*, each with its own program stack.
- At strategic points in their algorithms they call a function, `swtch()`, which puts the execution of the current task "on hold" and resumes execution of the other task.

Think of a board game such as Snakes and Ladders with two pieces on the board. You move one piece along its path until it lands on a square that says "switch". You leave the piece exactly where it is, pick up the other piece and move *it* along its own path until it, too, reaches a "switch" square. Then you put it down and pick up the first piece again, which carries on from where it was left. Both pieces make progress round the board, although at any moment only one of them is moving. A chess clock captures the same rhythm: you make your move, press the button, and your opponent's clock starts. A task that gives up the CPU in this way is said to *yield* it, and because the tasks take turns only when they choose to, the technique is called *cooperative multitasking*. Functions that suspend themselves in the middle and are later resumed from the same point, rather than always starting at the top, are called *coroutines*.

History: Coroutines

The word *coroutine* was coined by Melvin Conway in 1958, while he was designing a compiler that processed a COBOL program in several passes without storing the intermediate results: each pass was written as a coroutine that handed its output, item by item, to the next. He described the idea in "Design of a separable transition-diagram compiler" (*Communications of the ACM*, 1963). Donald Knuth gave coroutines a section of their own in the first volume of *The Art of Computer Programming*, where the relationship between two coroutines is symmetric: neither is the master, and each calls the other as if it were the subroutine.

11.5.1 Execution context

To put a task on hold and resume it later, we need to understand exactly what "where a task is" consists of. When a program runs, the processor holds its working data in registers: the program counter says where the next instruction will be fetched from; the stack pointer marks the top of the stack, where the chain of active function calls keeps its return addresses, saved registers and local variables; and the other

registers hold whatever temporary values the program is working on at that moment. Everything else the program uses — its code, its global variables, its stack — is in memory, and will stay there untouched if the task is suspended (as long as the other task leaves it alone).

This leads to one of the central notions of operating systems:

Key idea

The *execution context* of a running program is the minimal amount of data — usually the contents of some processor registers — that must be saved to allow the program to be interrupted and, later, resumed. Saving one context and loading another is a *context switch*.

If we save a task's context somewhere in memory, let the CPU work on something else, and later load the saved values back into the registers, the CPU will carry on executing the task as if nothing had happened. It has the same program counter, so it fetches the next instruction from the same place; it has the same stack pointer, so it finds its local variables and return addresses where it left them; and it has the same values in every register that matters.

11.5.2 Which registers make up the context?

RISC-V has 32 integer registers and a program counter. Must `swtch` save all of them? The answer comes from a fact we can exploit: `swtch` is called like any other C function, and the caller has been compiled according to the RISC-V calling convention of Section 4.6 (Table 2.1). Recall its two groups of registers. A called function may freely overwrite the temporaries `t0`–`t6`, the argument registers `a0`–`a7` and `ra`, whereas it must return with `sp` and `s0`–`s11` exactly as it found them. The convention is a contract, and the compiler relies on it everywhere: a value that has to survive a function call is kept in an `s` register or on the stack, never in a `t` or `a` register.

Now consider a task that calls `swtch`. From its point of view, `swtch` is just a function: it is called, and some time later it returns. That the other task ran in between is invisible — *provided* that, when `swtch` returns, the task finds every register the contract promises to preserve exactly as it left it. So the context that `swtch` must save and restore consists of

- the callee-saved registers `s0`–`s11` and the stack pointer `sp`, because the contract says they survive a call; and
- the return address `ra`, not because the contract requires it, but because it is where the task will continue.

The temporaries and argument registers need not be saved at all: the task's own compiled code has already assumed that `swtch` will destroy them.

What about the program counter? It is never saved explicitly, and it does not need to be. When a task calls `swtch`, the `jal` instruction puts the address of the instruction after the call into `ra`. That *is* the point at which the task should resume. If `swtch` saves `ra` with the rest of the context, and later restores it and executes `ret` (which jumps to the address in `ra`), the task continues from the instruction after its call to `swtch` — which is precisely what returning from a function means. The return address stands in for the program counter.

11.5.3 The context switch, step by step

Figure 11.3 shows two tasks taking turns. Task 1 is running. At some point it calls `swtch`, which saves the processor registers of task 1 into task 1's context in memory, loads the registers of task 2 from task 2's context, and returns. But it does not return to task 1: the values in `ra` and `sp` now belong to task 2, so the return lands in task 2, at the point where task 2 last called `swtch`. Task 2 runs until it, in turn, calls `swtch`, which saves task 2's registers, restores task 1's, and "returns" into task 1 — which continues from its call as if it had been an ordinary function call. Each task goes into `swtch` and comes out of `swtch`; it is only that it comes out some time later, after the other task has had its turn.

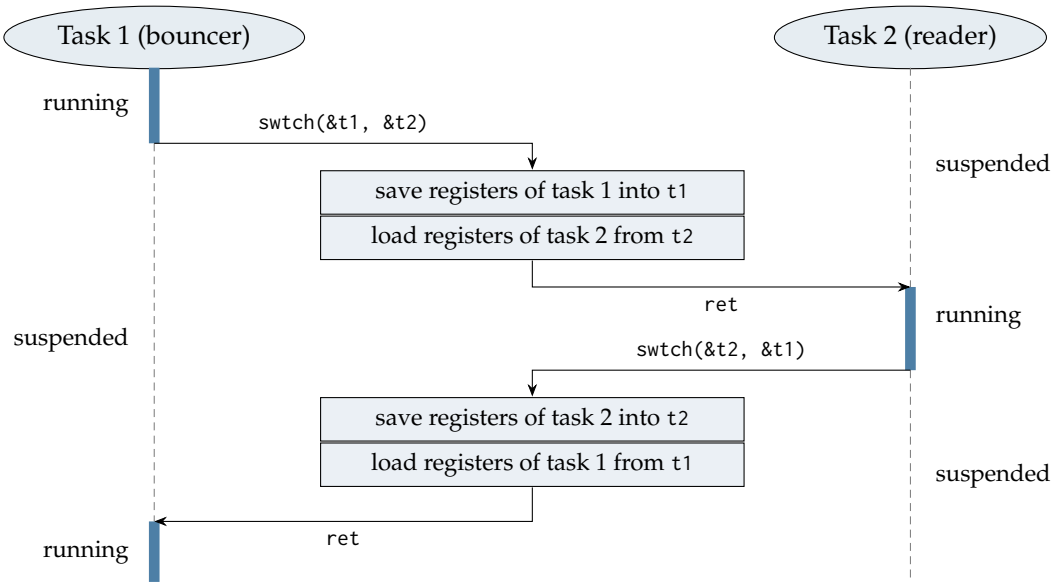


Figure 11.3: A context switch between two tasks. Each call to `swtch` saves the registers of the caller and returns into the other task. `t1` and `t2` stand for the two context structures in memory.

11.5.4 One stack per task

Why does each task need its own stack? Because each task has its own chain of active function calls. When the reader task is suspended, it is three calls deep: `reader_task` has called `readint`, which has called `swtch`. The stack frames of `reader_task` and `readint` hold their saved return addresses, the saved values of the `s` registers they use, and any local variables the compiler did not keep in registers. All of that must still be there when the reader resumes. If the bouncer used the same stack, its own function calls would push frames on top of the reader's — or, worse, at the very addresses the reader's frames occupy, since the bouncer's stack pointer would have no idea where the reader's frames ended. The first call that the bouncer made would overwrite the reader's saved return address.

So each task gets a separate region of memory for its stack, and its stack pointer — saved in its context while it is suspended — points into that region. Switching the stack pointer is what switches the whole chain of function calls from one task to the other. Together, the stack pointer and the return address are what make the trick work: `sp` selects *which* chain of calls we are in, and `ra` selects *where* in the innermost of them we continue.

11.6 Code: the context and `swtch`

The coroutine version of Pong adds two files to the project: a header, `coroutines.h`, that declares the context structure and the switching function, and an assembly-language file, `swtch.S`, that implements the switch. The function has to be written in assembly because no C statement can say "store register `s5` here" or "set the stack pointer to this value".

```
9  #pragma once
10
11 typedef struct context {
12         void *ra;
13         void *sp;
14         void *gp;
15         void *tp;
16         void *s0;
17         void *s1;
18         void *a0;
19         void *a1;
20         void *a2;
21         void *a3;
22         void *a4;
23         void *a5;
24         void *a6;
25         void *a7;
26         void *s2;
27         void *s3;
28         void *s4;
29         void *s5;
30         void *s6;
31         void *s7;
32         void *s8;
33         void *s9;
34         void *s10;
35         void *s11;
36 } context;
37
38 void swtch(context *current, context *next);
```

Listing 11.6: The execution context (`coroutines/coroutines.h`).

The structure has one field for each register that `swtch` saves. Each field is declared as `void *` simply because a pointer is a convenient 32-bit type: some of these registers hold addresses (`ra`, `sp`), others hold integers, and the C code that initialises a context (below) needs to store addresses of functions and of arrays into it without casts. On RV32 every field is four bytes, so the field number i (counting from 0) lies at byte offset $4i$ from the start of the structure: `ra` at 0, `sp` at 4, `gp` at 8, ..., `s11` at 92. The whole context is $24 \times 4 = 96$ bytes. The offsets matter, because the assembly code below uses them as numbers, and the structure and the assembly code must agree.

Comparing the fields with our analysis, you will notice that the structure saves more than it strictly needs. Besides `ra`, `sp` and `s0`–`s11` (14 registers), it saves `gp` and `tp`, which our programs never change, and the argument registers `a0`–`a7`, which the calling convention does not preserve. Saving them does no harm — it costs twenty extra instructions per switch — and saving a little too much is a safer mistake than saving too little. The prototype on line 38 tells us how `swtch` is called: the address of the context in which to save the current task, and the address of the context from

which to load the next one. By the calling convention these two pointers arrive in `a0` and `a1`.

```
1  # Context switching function
2  # void swtch(context *current, context *next)
3
4          .text
5          .globl swtch
6  swtch:
7          mv t0,a0
8          mv t1,a1
9
10         sw ra, 0(t0)
11         sw sp, 4(t0)
12         sw gp, 8(t0)
13         sw tp, 12(t0)
14         sw s0, 16(t0)
15         sw s1, 20(t0)
16         sw a0, 24(t0)
17         sw a1, 28(t0)
18         sw a2, 32(t0)
19         sw a3, 36(t0)
20         sw a4, 40(t0)
21         sw a5, 44(t0)
22         sw a6, 48(t0)
23         sw a7, 52(t0)
24         sw s2, 56(t0)
25         sw s3, 60(t0)
26         sw s4, 64(t0)
27         sw s5, 68(t0)
28         sw s6, 72(t0)
29         sw s7, 76(t0)
30         sw s8, 80(t0)
31         sw s9, 84(t0)
32         sw s10, 88(t0)
33         sw s11, 92(t0)
```

Listing 11.7: `swtch` saves the current context (`coroutines/swtch.S`, lines 1–33).

Lines 4–6.
The usual preamble (Section 3.7): the code goes in `.text`, and `.globl` lets the linker resolve the calls in `main.c` to the label `swtch`.

Lines 7–8.
Copy the two arguments into the temporaries `t0` and `t1`, and use those as base registers from now on. Why not use `a0` and `a1` directly? Because this version of the context includes `a0` and `a1` themselves, and lines 41–42 below load new values into them. Had we used `a1` as the base address for the loads, line 42 would overwrite it, and every load after that would read from the wrong place. `t0` and `t1` are free for us to use: they are temporaries, and they are not part of the context.

Lines 10–33.
Store each register into its field of the current context, at the offsets computed above. Line 10 is the most important of them: it saves the return address, the point in the calling task to which we will eventually come back. Line 11 saves the stack pointer, which identifies the task's stack.

At this point the current task has been completely recorded in memory. The second half of the function brings the other task back to life:

```
35          lw ra, 0(t1)
36          lw sp, 4(t1)
37          lw gp, 8(t1)
38          lw tp, 12(t1)
39          lw s0, 16(t1)
40          lw s1, 20(t1)
41          lw a0, 24(t1)
42          lw a1, 28(t1)
43          lw a2, 32(t1)
44          lw a3, 36(t1)
45          lw a4, 40(t1)
46          lw a5, 44(t1)
47          lw a6, 48(t1)
48          lw a7, 52(t1)
49          lw s2, 56(t1)
50          lw s3, 60(t1)
51          lw s4, 64(t1)
52          lw s5, 68(t1)
53          lw s6, 72(t1)
54          lw s7, 76(t1)
55          lw s8, 80(t1)
56          lw s9, 84(t1)
57          lw s10, 88(t1)
58          lw s11, 92(t1)
59
60          ret
```

Listing 11.8: `swtch` loads the next context and "returns" (`coroutines/swtch.S`, lines 35–60).

Lines 35–58.
Load every register from the next context, using the same offsets. Line 35 loads the next task's return address into `ra` and line 36 its stack pointer into `sp`. From line 36 on, the CPU is, in effect, on the other task's stack.

Line 60.
`ret` is the pseudo-instruction `jalr x0, 0(ra)`: jump to the address in `ra`. Since `ra` now holds the return address saved when the *other* task called `swtch`, the CPU continues in that task, just after its call. The call that was made by one task returns in another.

That is the whole of the context switch: 2 moves, 24 stores, 24 loads and a return — 51 instructions. Nothing in it knows anything about Pong, about dots or keyboards; it would switch between any two pieces of C code.

11.6.1 What the saved registers are protecting

It is worth seeing, on the real program, why the `s` registers have to be saved. When I compile the coroutine version with its Makefile (GCC 13.2 at optimisation level `-O`), the compiler keeps the bouncer's velocity in callee-saved registers: `dx` in `s0` and `dy` in `s1`, with the dot colour `0xFFFFFF` in `s6`. That is a natural choice — they are needed in every iteration of the loop, and the loop calls `swtch`, so they must be in registers that

survive a call. Meanwhile, `readint` in the reader task keeps the address of `RCR` in `s0`, the address of `TTY` in `s1`, and the number being accumulated, `res`, in `s6`, for the same reason.

So the same register `s0` means "the dot's horizontal speed" in one task and "the address of the keyboard status register" in the other. If `swtch` did not save and restore it, the bouncer would come back from a switch with its speed set to `0xFFFF0004`, and the dot would leap off the screen; the reader would come back and try to read the keyboard status from whatever address the bouncer's speed happened to make it. Saving the `s` registers in the context gives each task its own private copy of them. Figure 11.4 shows the state of memory and registers on either side of a switch from the bouncer to the reader.

11.7 Code: starting the tasks

A context is saved by `swtch` when a task is suspended. But the very first time we switch to a task, there is no saved context to restore — the task has never run. We have to make one by hand, so that `swtch` can "resume" the task at its very beginning. Here is how `main` does it:

```
39 /* global (UNINITIALIZED!) task contexts for dot bouncer and coordinate reader*/
40 context main_context;
41 context bouncer_context;
42 context reader_context;
43
44 /* program stacks for dot bouncer and coordinate reader tasks */
45 int bouncer_stack[256];
46 int reader_stack[256];
47
48 void bouncer_task();
49 void reader_task();
50
51 void main()
52 {
53    bouncer_context.sp = &(bouncer_stack[255]); // points to the last element in
       bouncer stack
54    bouncer_context.ra = bouncer_task;
55    reader_context.sp = &(reader_stack[255]); // points to the last element in
       bouncer stack
56    reader_context.ra = reader_task;
57    swtch(&main_context, &bouncer_context); // start bouncer task
58 }
```

Listing 11.9: Creating and starting the two tasks (`coroutines/main.c`).

Lines 40–42.
Three contexts. Two are for our tasks; the third, `main_context`, is somewhere for `swtch` to save the registers of `main` itself when it starts the first task, since `swtch` always saves before it loads.

Lines 45–46.
Two stacks of 256 words (1 KB) each, ordinary global arrays in RAM. Nothing makes them stacks except that we are about to point stack pointers at them.

Lines 53 and 55.
Point each task's saved `sp` at the last element of its array. RISC-V stacks grow

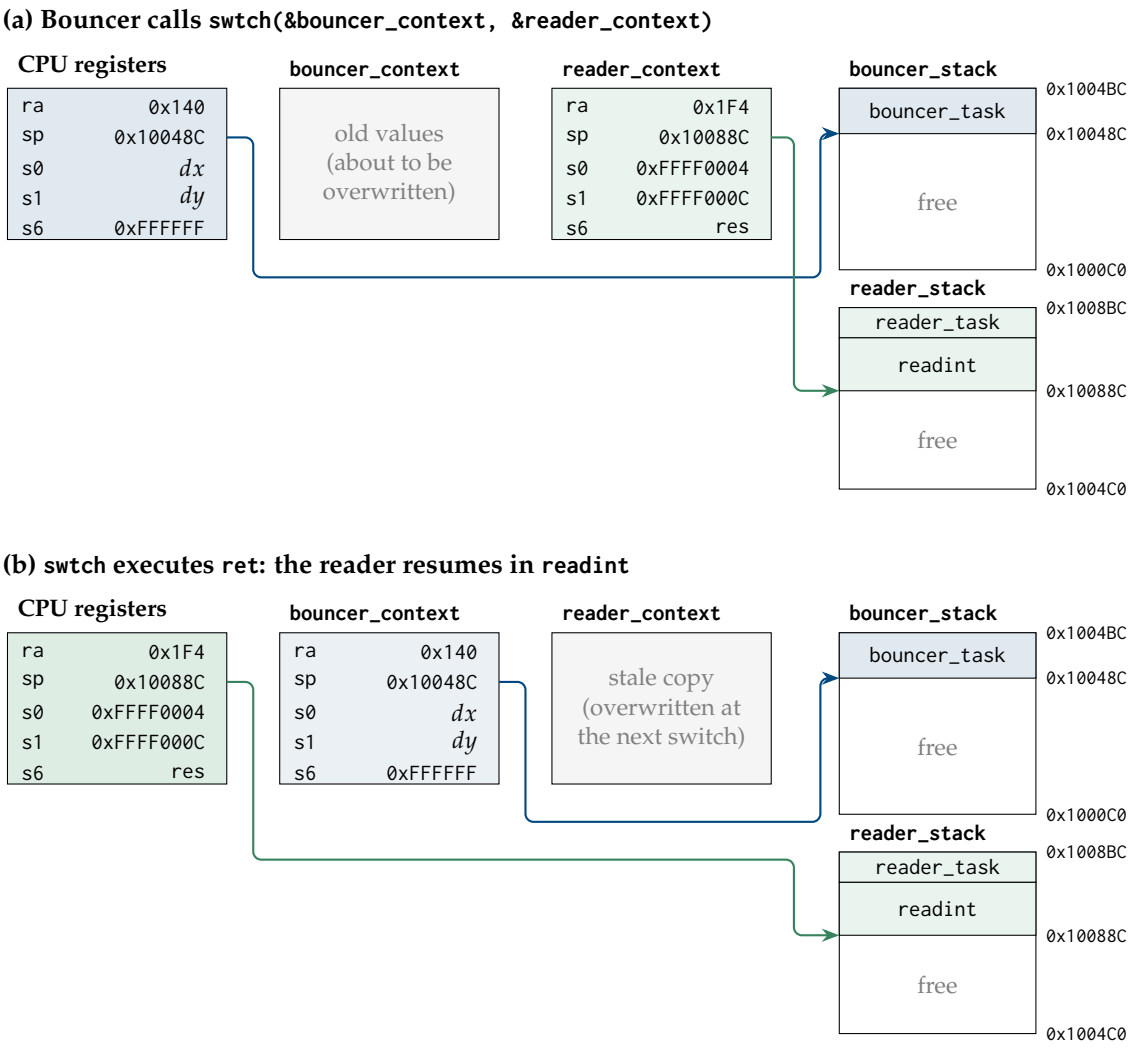


Figure 11.4: Registers, contexts and stacks (a) when the bouncer calls `swtch` and (b) when `swtch` returns into the reader. Only five of the 24 saved registers are shown. Arrows show where each saved or live stack pointer points; higher addresses are at the top of each stack. `0x140` is the instruction after the bouncer's call to `swtch`, `0x1F4` the instruction after the reader's call in `readint`. Addresses and register assignments are from my build (GCC 13.2, `-O`); another compiler may choose differently.

towards lower addresses, so the stack must start at the high end of the array. The first thing the task's function does is subtract the size of its frame from `sp`, and its frame then lies within the array. (The comment on line 55 is copied from line 53; it means the reader's stack, of course.)

Lines 54 and 56.
Set each task's saved `ra` to the address of the task's function. In C, the name of a function used as a value is its address.

Line 57.
Switch from `main` to the bouncer. `swtch` saves `main`'s registers into `main_context`, loads the bouncer's context — the stack pointer at the top of `bouncer_stack`, and `ra` equal to the address of `bouncer_task` — and executes `ret`. The "return" therefore jumps to the first instruction of `bouncer_task`, with the stack pointer at the top of an empty stack. To `bouncer_task`, this looks exactly as though it had just been called.

The remaining 22 fields of the two task contexts are never initialised, so the first switch into each task loads whatever happens to be in RAM into `gp`, `tp`, the `a` registers and the `s` registers. That is harmless. A function that has just been called makes no assumption about the `s` registers — it will save them before using them, and restore those values when it returns — nor about the `t` registers, and it reads `a0`–`a7` only for its arguments, of which our task functions have none. Only `sp` and `ra` must be right. (If you wanted to pass an argument to a task, you could set `a0` in its context before the first switch; Exercise 8. asks you to try.)

Nothing ever switches back to `main_context`, so `main` never returns: the `swtch` on line 57 is a one-way door. That is fine; on our bare-metal computer there is nowhere for `main` to return to anyway, as `crt0` would only loop forever.

History: Why `swtch`?

The odd spelling is a tradition. The function that switched processes in the Sixth Edition of Unix (1975) was called `swtch()`, because `switch` is a reserved word in C.

A note on alignment: the RISC-V calling convention asks for the stack pointer to be a multiple of 16 whenever a function is called, and `&bouncer_stack[255]` is not (it is `0x1004BC` in our build). Nor, for that matter, is the initial stack pointer set by `crt0`, which the linker script places at the last word of the 64 KB of RAM it declares, `0x0010FFFC`. On our RV32IM computer nothing depends on the alignment, so the programs work; code that uses 64-bit or floating-point values on the stack, or calls library functions compiled with the assumption, may not. Exercise 6. returns to this.

11.8 Code: the bouncer and the reader

With `swtch` available, each task can be written in the straightforward, blocking style. The function `bouncer_task` is the body of the blocking version's `main` without the input processing: the same initial position, the same velocities, the same five-step loop. The only change is at the end of the loop.

```
104         x = new_x;
105         y = new_y; // remember new coordinates
106 
107         if (*RCR != 0)
108         {
109             swtch(&bouncer_context, &reader_context);
110         }
111     }
112 }
```

Listing 11.10: The end of the bouncer's loop (`coroutines/main.c`).

After each step of the dot, the bouncer checks the keyboard. If a key is waiting, it yields the CPU to the reader. The reader is the blocking input algorithm, with one addition inside `readint`:

```
114 /* read integer from keyboard */
115 int readint()
116 {
117     int res, key;
118     for (res = 0;;)
119     {
120         while (*RCR == 0)
121         {
122             swtch(&reader_context, &bouncer_context);
123         };
124         key = *RDR;
125         *TTY = key;
126         if ((key < '0') || (key > '9'))
127             break;
128         res = res * 10 + (key - '0');
129     }
130     return res;
131 }
132 
133 void reader_task()
134 {
135     int new_x, new_y;
136     new_x = readint();
137     new_y = readint();
138     if ((new_x >= VGA_WIDTH) || (new_y >= VGA_HEIGHT))
139     {
140         printstr("New coords out of bounds!\n");
141     }
142     else
143     {
144         VGA_PLOT(x, y, 0x000000);           // erase the dot by making it black
145         VGA_PLOT(new_x, new_y, DOT_COLOUR); // paint it in new coordinates
146         x = new_x;
147         y = new_y;
148     }
149 }
```

Listing 11.11: The reader task (`coroutines/main.c`).

Compare `readint` with Listing 11.2. The empty waiting loop has acquired a body: instead of spinning while no key is available, the reader hands the CPU to the bouncer, which moves the dot one step and then, if a key is available, hands it

back. `swtch` is called from inside `readint`, which is called from two different places in `reader_task`, and it does not matter: `swtch` returns into whichever call of `readint` was suspended, with its `res` intact, because the call chain on the reader's stack is preserved. `reader_task` itself is the input algorithm as we first stated it, line for line. Neither algorithm had to be turned inside out.

11.8.1 Following a keystroke

Let us follow the program as the user types `7`, `3` and Enter.

1. `main` switches to the bouncer, which plots the dot and prints the instructions. No key is waiting, so the bouncer goes round its loop again and again, without ever calling `swtch`. The reader has not started yet.

2. The user types `7`. At the end of its next step the bouncer sees `*RCR != 0` and calls `swtch(&bouncer_context, &reader_context)`. The bouncer's registers are saved, and the reader's context, as prepared by `main`, is loaded. The reader starts at the top of `reader_task` and calls `readint`.

3. In `readint` a key is waiting, so the `while` loop does not execute; the `7` is read, echoed and accumulated: `res` is 7. Back at the top of the `for` loop, the keyboard is now empty, so the reader calls `swtch(&reader_context, &bouncer_context)`.

4. The bouncer resumes from *its* call to `swtch`, at the end of its loop, and goes on moving the dot. The reader is frozen inside `readint`, with `res` = 7 in its saved `s6`.

5. The user types the comma. The bouncer switches to the reader, which returns from its `swtch` inside the `while` loop, finds a key, reads the comma and returns 7 from `readint`. `reader_task` stores it in `new_x` and calls `readint` again, which finds the keyboard empty and yields.

6. The same happens for `3` and for Enter. When Enter arrives, the second `readint` returns 3, and `reader_task` checks the coordinates, moves the dot to $(7, 3)$ and returns.

What happens when `reader_task` returns? It was never called, so there is no caller to return to. Its `ret` jumps to the address in `ra`, and the value of `ra` when the function was entered was the one `main` put in the context: the address of `reader_task` itself. The function saved that value on entry and restores it, along with the stack pointer, on exit; so "returning" jumps back to the start of `reader_task`, with the stack pointer back at the top of `reader_stack`. The reader simply starts again, calls `readint`, finds no key and yields. The program works — but it works by a happy accident of how we started the task, and a clearer version would wrap the body of `reader_task` in an explicit `for (;;)` loop, as the bouncer's is.

11.8.2 Where to put the switches

Run this version in Logisim and compare its speed with the other two. It is the only version whose cost you can see changing as you type: the dot moves as fast as in the blocking version while you are idle, and slows down noticeably while you are typing. The reason is the cost of the context switch.

Counting the instructions in the compiled code, one step of the bouncing loop takes about twenty instructions. A single switch takes 51 instructions in `swtch` alone,

plus a few to set up the call; and while the user is typing, each step of the dot involves *two* switches, there and back. In my first version of this program the bouncer called `swtch` unconditionally at the end of every step, so that the reader could check the keyboard; then every step cost about 130 instructions instead of 20, and the dot moved about six times more slowly, whether you were typing or not. Most of those switches were wasted: the reader woke up, found no key, and switched straight back. Adding the test `if (*RCR != 0)` around the bouncer's call means that the bouncer only wakes the reader when the reader has something to do.

This is the general lesson: in cooperative multitasking, *calls to* `swtch` *must be placed strategically*. Too few, and a task hogs the CPU while the others wait; too many, and the CPU spends its time saving and loading registers instead of doing useful work. Notice, too, what the test `*RCR != 0` amounts to: the bouncer knows what the reader is waiting for, and does not resume it until that has happened. Operating systems do the same on a grand scale. A process waiting for input is marked as *sleeping*, and the scheduler does not waste time switching to it until the input has arrived and the process has been *woken up*. We will meet this in Chapter 14.

11.9 The limits of cooperation

To sum up the coroutine approach:

Pros: Both tasks run quasi-simultaneously, and both algorithms are easy to read and understand.

Cons: Context switching takes a lot of time, and calls to `swtch()` need to be placed strategically to minimise the amount of unnecessary context switching.

The readability is the real prize. We described two algorithms separately, each in its natural form, and made them run at the same time by adding a handful of calls. That is why so many programming languages give you threads: a program made of independent tasks is easier to write and to understand than one big state machine.

There is a second, quieter advantage. Because a task can only lose the CPU when it calls `swtch`, everything it does between two calls is *indivisible* as far as the other task is concerned. The bouncer reads `x` and `y`, computes the new position and writes it back, and the reader cannot change `x` and `y` halfway through, because the reader is not running. The shared data that worried us at the start of the chapter is safe here, without any special precautions. That will no longer be true once the switching is done by hardware.

The weaknesses go deeper than speed, and they all follow from the word *cooperative*. The scheme works only if every task plays fair.

- *A task that does not yield freezes everything.* Suppose the reader's waiting loop had been left as it was in Listing 11.2, with an empty body. The first time the reader ran out of keys it would spin in that loop for ever, and the dot would never move again — the bouncer cannot take the CPU back, because only the running task can call `swtch`. A bug that produces an infinite loop in any one task has the same effect, and so does a task that simply has a long computation to do and does not think to yield in the middle of it.

- *Every waiting point must be rewritten.* A cooperative task may never wait in the usual way, by spinning on a device register; every such loop must contain a yield. A library function that waits for input internally, written by someone who did not know about our `swtch`, will block the whole program.

- *Response time is not guaranteed.* How quickly the system reacts to a key press depends on how long the currently running task takes to reach its next `swtch`. If the bouncer's step took a second, the reader would wait up to a second for its turn, however urgent the key.

- *The tasks must trust each other.* Nothing stops one task from overwriting the other's stack, its context or its variables. Here both tasks are parts of one small program written by one person, so that is acceptable; for unrelated programs written by different people, it is not.

All four problems have the same root: the decision to switch is taken by the running task. What we would like is for something *outside* the tasks to be able to stop a task in its tracks — when a key is pressed, when a timer expires — save its context and switch to another, whether or not the running task is willing. The hardware mechanism that makes this possible is the *interrupt*, and it is the subject of the next chapter. With interrupts and a timer, the same idea of saving one context and loading another becomes *preemptive multitasking*, in which a task can be switched out between any two instructions. That is the subject of Chapter 13, and it is what every general-purpose operating system does.

One difference is worth noticing already. Because a task calls `swtch` voluntarily, as a function, the calling convention lets us leave out the temporary and argument registers from its context. An interrupt can strike between *any* two instructions, when a task may have a half-finished calculation in `t3` or an argument in `a2`; the code that switches tasks on an interrupt has no such contract to rely on, and must save every register. That is why the logOS trap handler of Section 7.12 saves all 31 general-purpose registers, together with the interrupted program counter, in its trap frame.

11.10 Real world

Coroutine switching in real systems. Our `swtch` is a small instance of a very common pattern. The Linux kernel does the same job in its architecture-specific `__switch_to` routine: on RISC-V, when the kernel switches from one thread to another, it saves the same callee-saved registers, plus the return address and stack pointer, into the outgoing thread's `thread_struct` and loads them from the incoming one's. The trick of switching stacks and "returning" into another thread is the same. In user space, POSIX once provided `getcontext`, `makecontext` and `swapcontext` for exactly this purpose — `swapcontext(&current, &next)` is our `swtch` with a different name — and although they were removed from the standard in POSIX.1-2008, glibc still supports them. Windows provides *fibers*, user-level tasks that are switched explicitly with `SwitchToFiber`.

Cooperative operating systems. For years, mainstream personal computers multitasked cooperatively. On the classic Mac OS, from MultiFinder in 1987 until Mac OS 9, applications shared the processor by calling the system's event functions (such as `WaitNextEvent`), which gave the system the chance to run another application. In 16-bit Windows (up to Windows 3.x), Windows applications likewise yielded whenever they asked the system for their next message. In both systems, a single application that went into a long computation without yielding — or crashed in a loop — froze the whole desktop, which is precisely the weakness we found in

Pong. Preemptive multitasking arrived for 32-bit applications with Windows 95 and Windows NT, and on the Mac with Mac OS X in 2001.

Event loops, async and await. The software state machine of Section 11.4 is alive and well. JavaScript in a web browser runs on a single thread: the browser calls your event handlers one at a time from an event loop, and each handler must return quickly, because while it runs nothing else on the page can happen — a handler stuck in a loop freezes the page, just as a task that never yields froze Pong. Programs that must wait for something in the middle of an algorithm are written either as chains of callbacks, which is the state-machine style, or with the `async` and `await` keywords found in JavaScript, Python, C# and other languages. An `await` looks like a call to `swtch`: the function is suspended and resumed later from the same point. Under the hood, the C# compiler implements it by turning the `async` function into a state machine — the very rewrite of Section 11.4, done automatically — and tools that translate modern JavaScript for older browsers do the same. Generators (`yield` in Python and JavaScript), and coroutines in Lua, Kotlin and C++20, are variations on the same idea. Go's lightweight goroutines were originally switched only at function calls and other safe points, a largely cooperative scheme; since Go 1.14 (2020) the runtime can also preempt a goroutine that runs a long loop without calls.

Embedded systems. Small microcontroller programs are very often built as a *super-loop*: one endless loop that calls each task's step function in turn, with each task written as a state machine — our Approach 2. Where memory is too scarce to give every task its own stack, *protothreads* (Adam Dunkels and colleagues, 2006, used in the Contiki operating system) let you write a task in sequential style, with macros that expand into a `switch` statement: a task can wait in the middle of its code, but its local variables are not preserved across the wait, because there is no separate stack to hold them. Larger embedded systems use a real-time operating system with preemptive, priority-based scheduling; NASA's Mars rovers, for example, run the VxWorks real-time operating system.

11.11 Exercises

1. *Execution context.* In your own words, explain why `swtch` does not need to save `t0`–`t6`, why it must save `s0`–`s11` and `sp`, and why it must save `ra` even though the calling convention does not require a function to preserve it. Which of the 24 fields of `context` could be removed without breaking the program?

2. *The state machine.* For the FSM version, the user types `1`, `2`, `,`, `4`, Enter, one key at a time, slowly enough that the game loop goes round several times between keys. Make a table with one row per key, showing the value of `input_state`, `inp_x` and `inp_y` after the key has been processed. Which keys are processed by "falling through" from one `case` to the next? What happens if the user types `40`,`4`?

3. *Tracing a switch.* In the coroutine version, the reader is suspended inside its *second* call of `readint`, waiting for the second digit of y. The bouncer has just found a key waiting and is about to call `swtch`. Describe the contents of the reader's stack, the fields `ra` and `sp` of both contexts, and the registers `ra` and `sp`, (a) just before the `jal` that calls `swtch` and (b) just after its `ret`. Use the disassembly file `pong-coroutines.asm` produced by `make` to find the actual addresses in your build.

4. *The cost of switching.* Remove the `if (*RCR != 0)` test in `bouncer_task`, so that the bouncer switches to the reader after every step. Predict how many instructions one step of the dot now costs, using the disassembly, and compare the speed of the dot in Logisim with the original. Then explain why the reader, as written, hardly ever finds the keyboard empty when it is resumed in the original version.

5. *A smaller context.* Remove `gp`, `tp` and `a0`–`a7` from the `context` structure and from `swtch.S`, adjusting all the offsets. Rebuild and check that Pong still works. How many instructions does a switch now take? Are lines 7–8 of `swtch.S` still needed?

6. *Stack overflow.* Using the linker map file `pong-coroutines.map`, draw the layout of the global variables in RAM. If the reader's stack overflowed, which variable would it overwrite first? Reduce both stacks to 16 words and see what happens. Then change `main` so that each initial stack pointer is the address just past the end of its array, rounded down to a multiple of 16, and explain why that is a better choice than `&stack[255]`.

7. *A task that returns.* Explain, step by step, what happens when `reader_task` returns. What would happen if `bouncer_task` returned? Rewrite `reader_task` with an explicit endless loop.

8. *Passing an argument.* Change `bouncer_task` to take the colour of the dot as a parameter, `void bouncer_task(int colour)`, and make `main` pass the colour by setting a field of `bouncer_context` before the first switch. Which field, and why does it work only for the first switch into the task?

9. *A third task.* Add a second bouncing dot of a different colour, as a third task with its own context and stack. Arrange the switches so that the three tasks take turns in a fixed order (*round robin*). You will find it convenient to keep the contexts in an array and to write a function `yield()` that switches from the current task to the next one. What must `yield()` remember between calls?

10. *Design.* A colleague proposes to make Pong robust against a task that never yields by having the bouncer check, every so often, whether the reader has been running too long. Explain why this cannot work in a cooperative system. What would the hardware have to provide to make it work? (Keep your answer; we will come back to it in the next chapter.)